Mahidol University
Faculty of Information and Communication Technology

Faculty of Information and Communication Technology
ITCS223 Introduction to Web Development
MYKEA

Project Phase II

Summitted by

Mr. Kanok	Suriyachan	6688119
Mr. Chatchanun	Toungpornsup	6688133
Mr. Supawit	Sirikulpiboon	6688166
Mr. Waris	Sirivarint	6688217
Mr. Thanawat	Thanasirithip	6688226

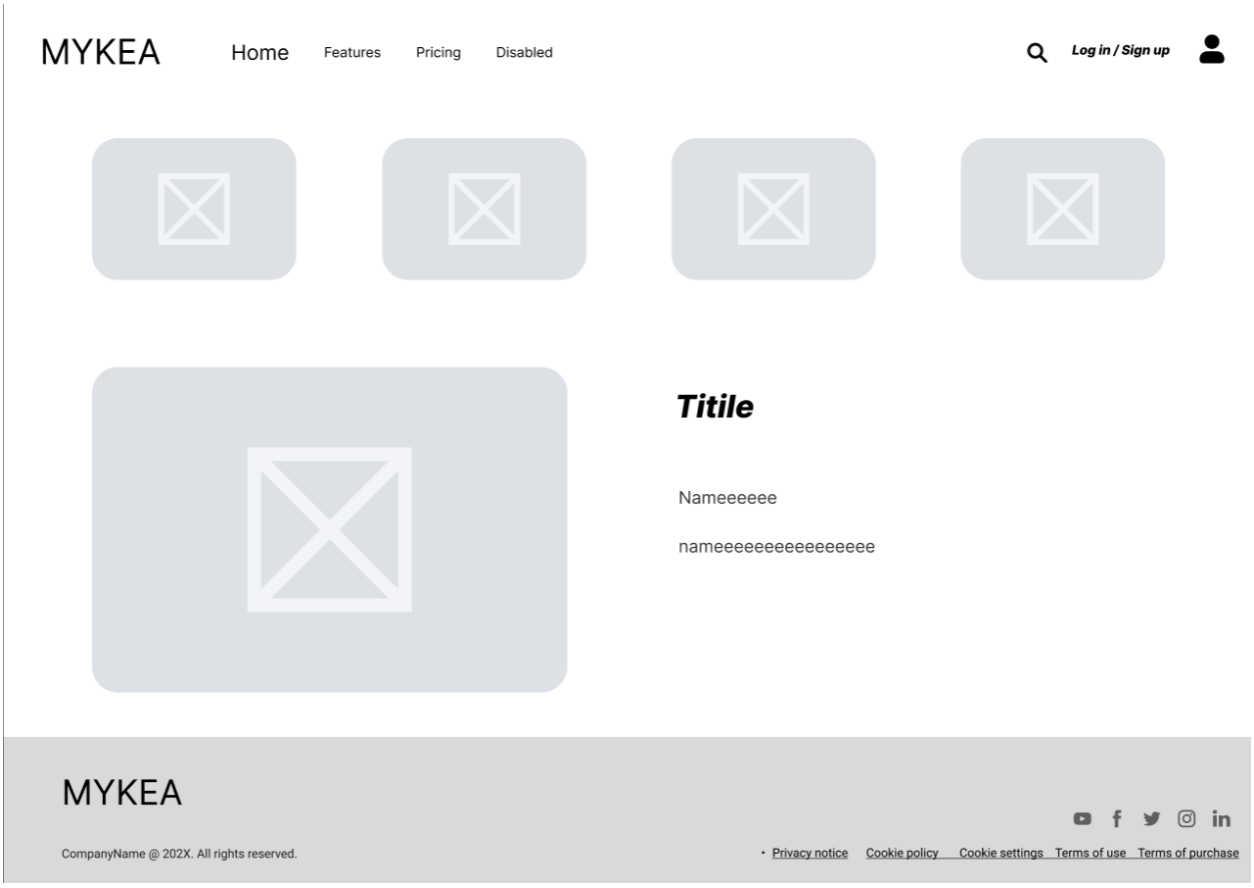
Presented to

Lect. Dr. Wudhichart Sawangphol, Lect. Asst. Prof. Dr. Jidapa Kraisangka and Lect. Dr. Pisit Praiwattana
Mahidol University
ITCS223 Introduction to Web Development
26 April 2025

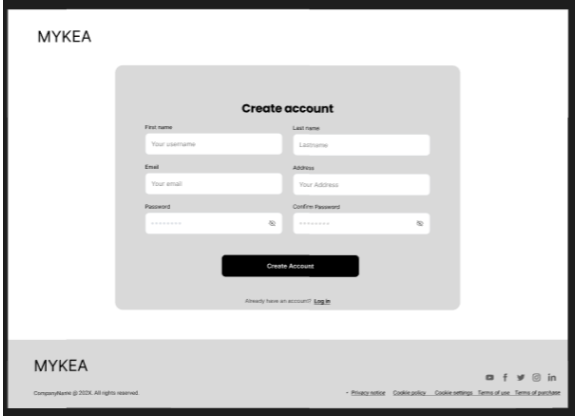
Business Domain

Our business is a Bed and accessories store which focuses on selling bed and bedding from various brands such as Boll & Branch, Coyuchi, Quince, Sijo, Buffy and many more. For our services, we have a website that customers can use for purchasing our products online. Our website contains many features such as a main page for welcoming customers, a log-in page for our customers to keep their purchasing information, product page to show details of our products, search page for customers to search for their products and admin page for our admin to edit the product information, adding and removing products from the website.

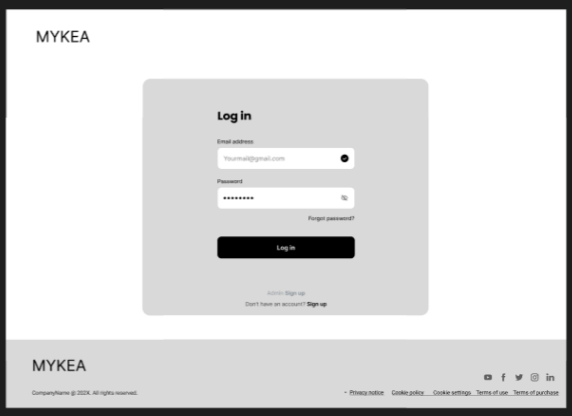
Wireframe


MYKEA
Home
Features
Pricing
Disabled
Log in / Sign up
Titile
Nameeeeee
nameeeeeeeeeeeeeeee
MYKEA
CompanyName @ 202X. All rights reserved.
Privacy notice
Cookie policy
Cookie settings
Terms of use
Terms of purchase

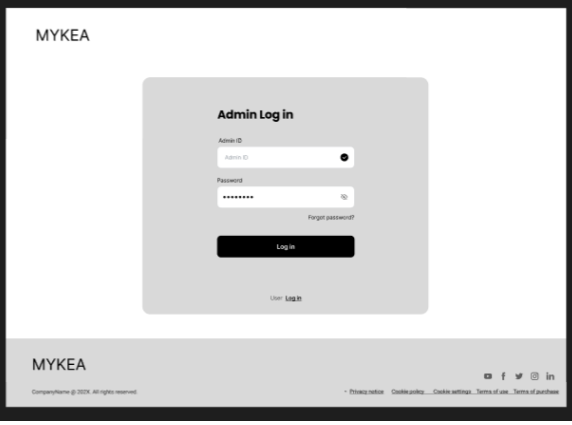
Creat account

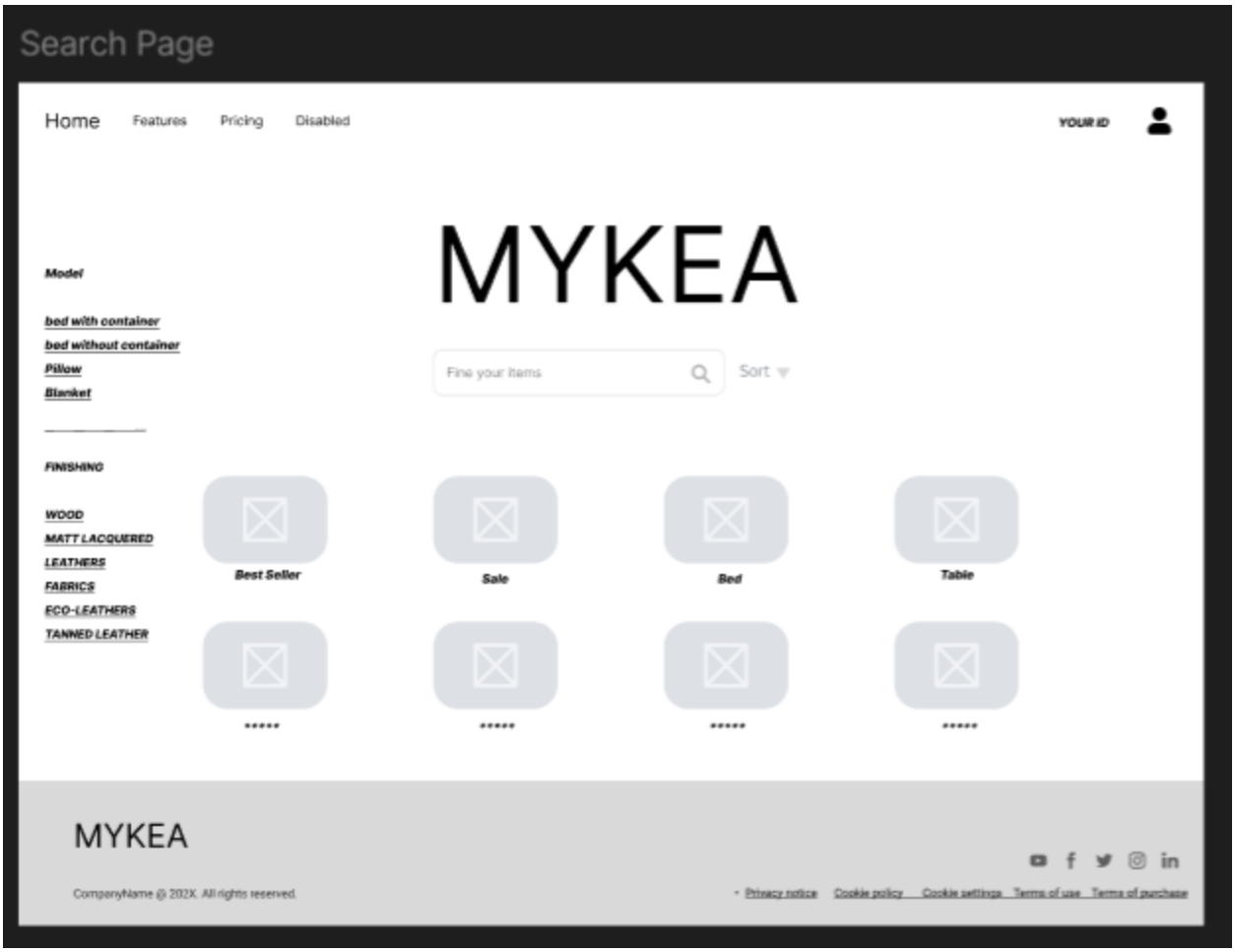


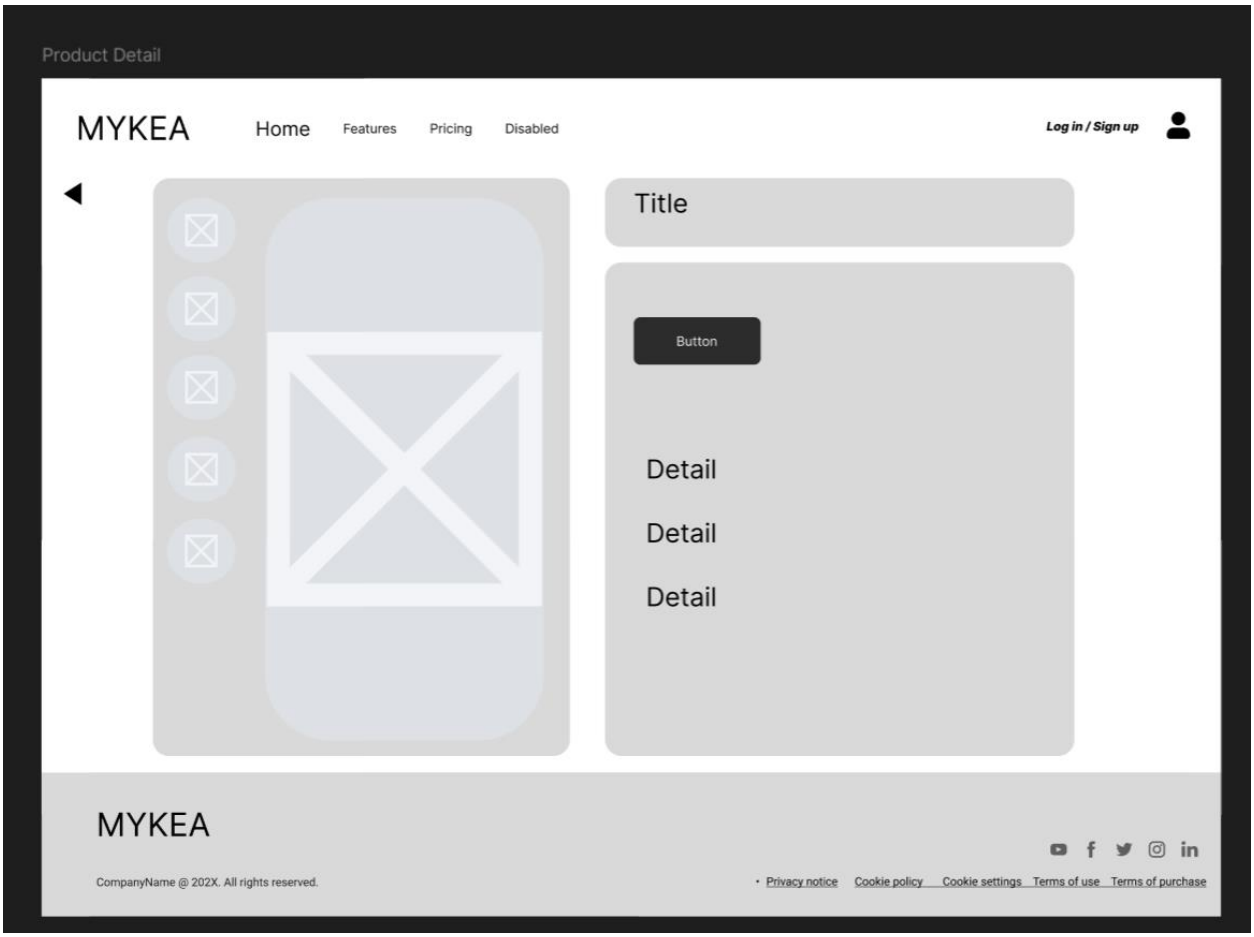
Login

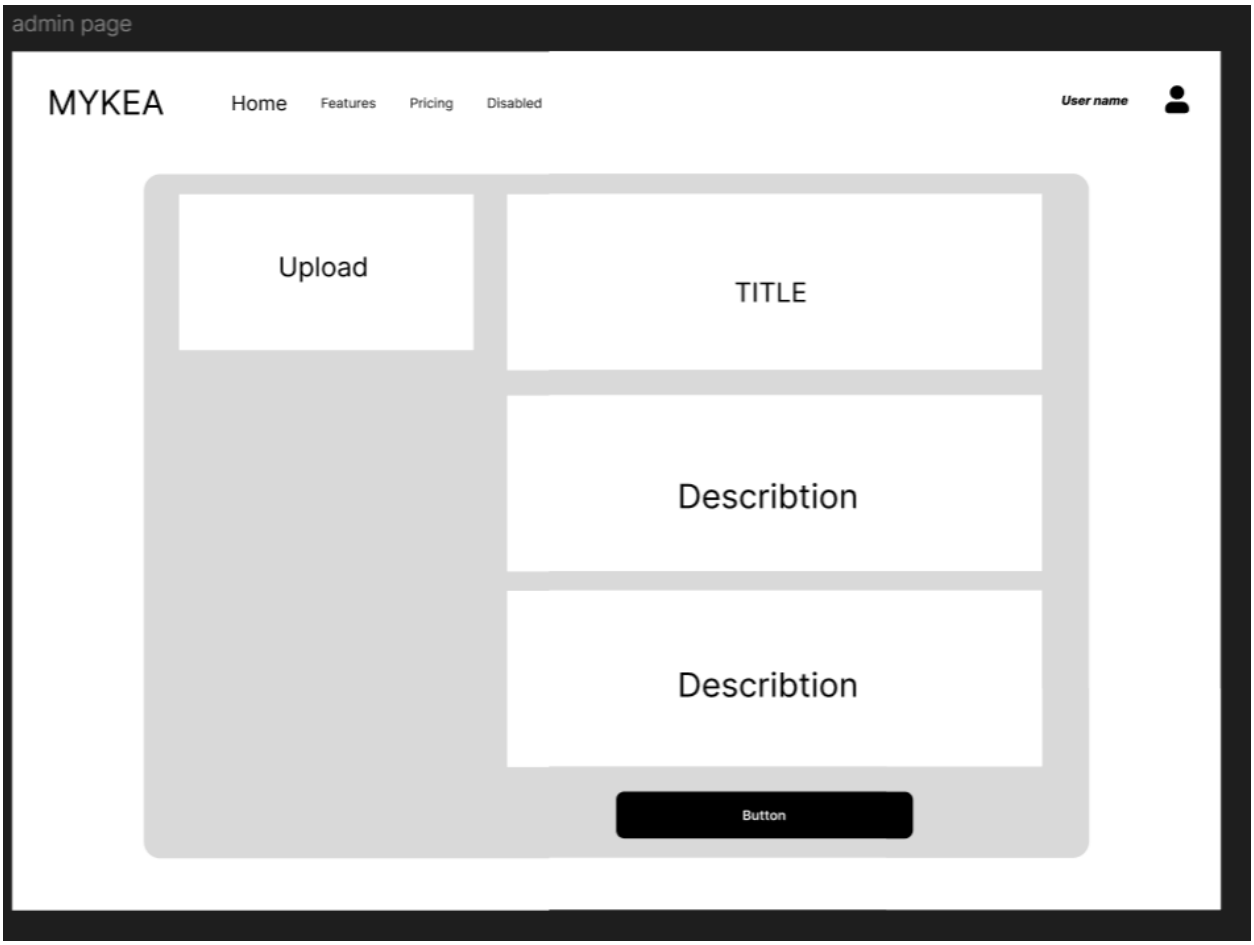


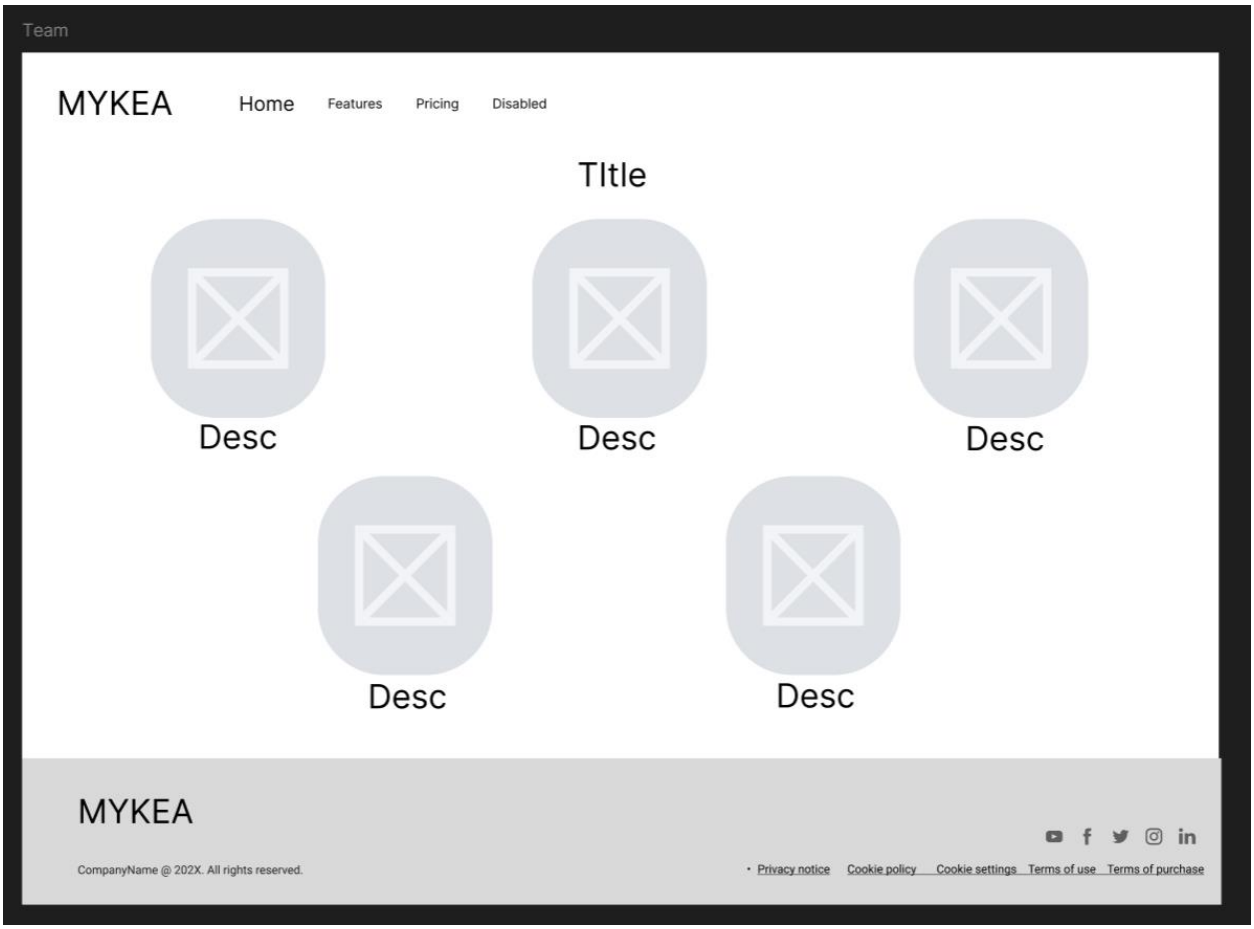
Admin Login

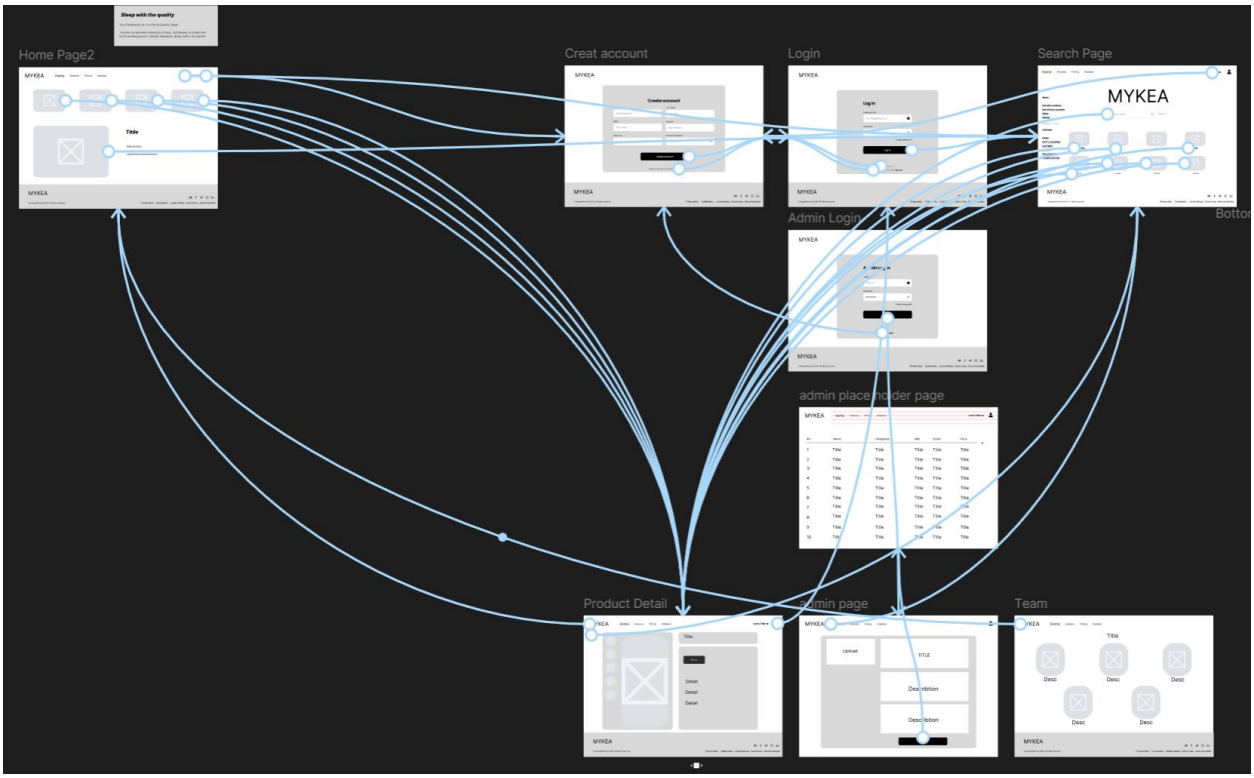



Search Page
Home
Features
Pricing
Disabled
YOUR ID
MYKEA
Model
bed with container
bed without container
Pillow
Blanket
Fine your items
Sort
FINISHING
WOOD
MATT LACQUERED
LEATHERS
FABRICS
ECO-LEATHERS
TANNED LEATHER
Best Seller
Sale
Bed
Table
MYKEA
CompanyName @ 202X. All rights reserved.
Privacy notice
Cookie policy
Cookie settings
Terms of use
Terms of purchase


Product Detail
MYKEA
Home
Features
Pricing
Disabled
Log in / Sign up
Title
Button
Detail
Detail
Detail
MYKEA
CompanyName @ 202X. All rights reserved.
Privacy notice
Cookie policy
Cookie settings
Terms of use
Terms of purchase

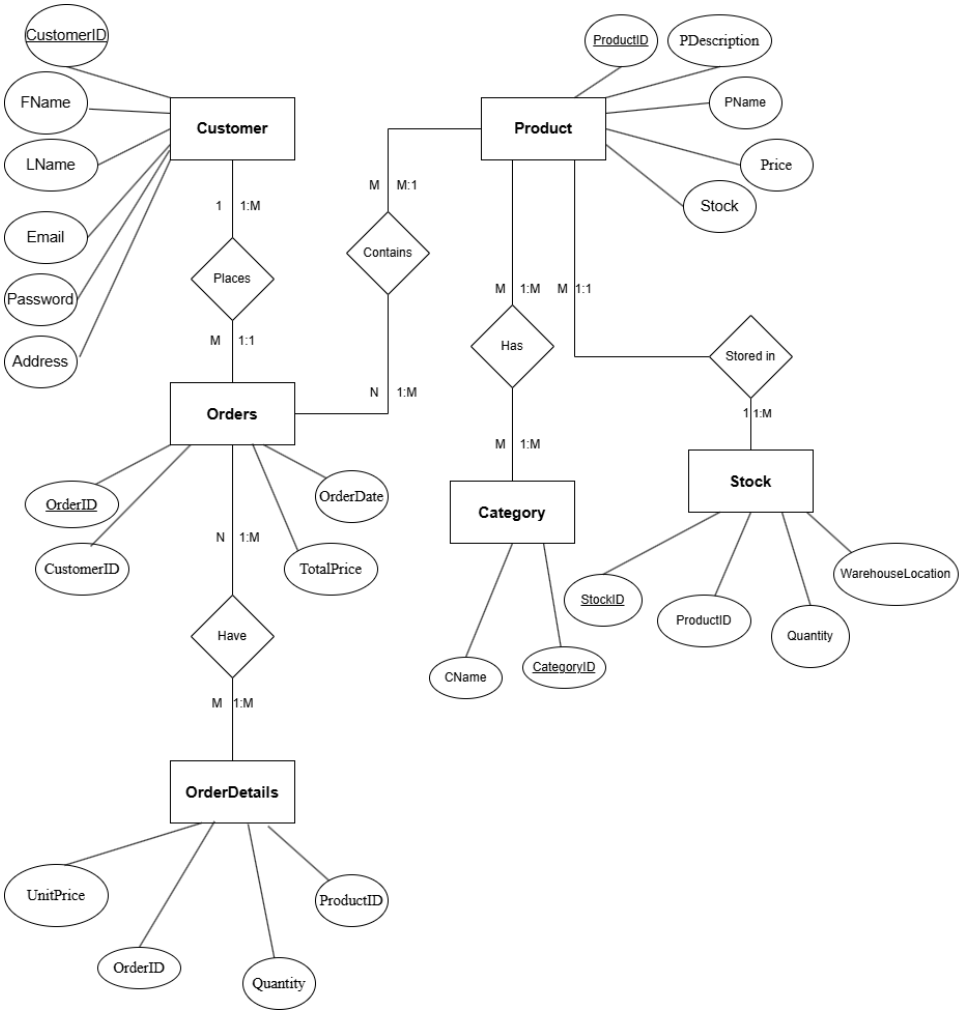

admin page
MYKEA
Home
Features
Pricing
Disabled
User name
Upload
TITLE
Describtion
Describtion
Button

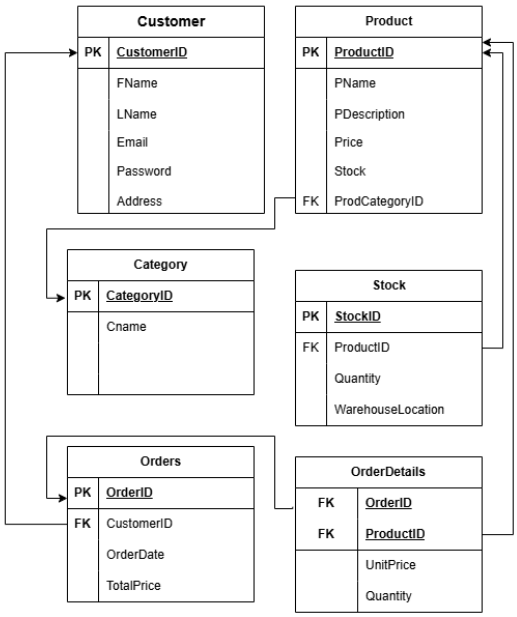

Team
MYKEA
Home
Features
Pricing
Disabled
TItle
Desc
Desc
Desc
Desc
Desc
MYKEA
CompanyName @ 202X. All rights reserved.
Privacy notice
Cookie policy
Cookie settings
Terms of use
Terms of purchase


Home Page2
Creat account
Login
Search Page
MYKEA
Admin Login
admin place holder page
Product Detail
Admin page
Team

Data Model

Entity Relationship Diagram & Relational Schema



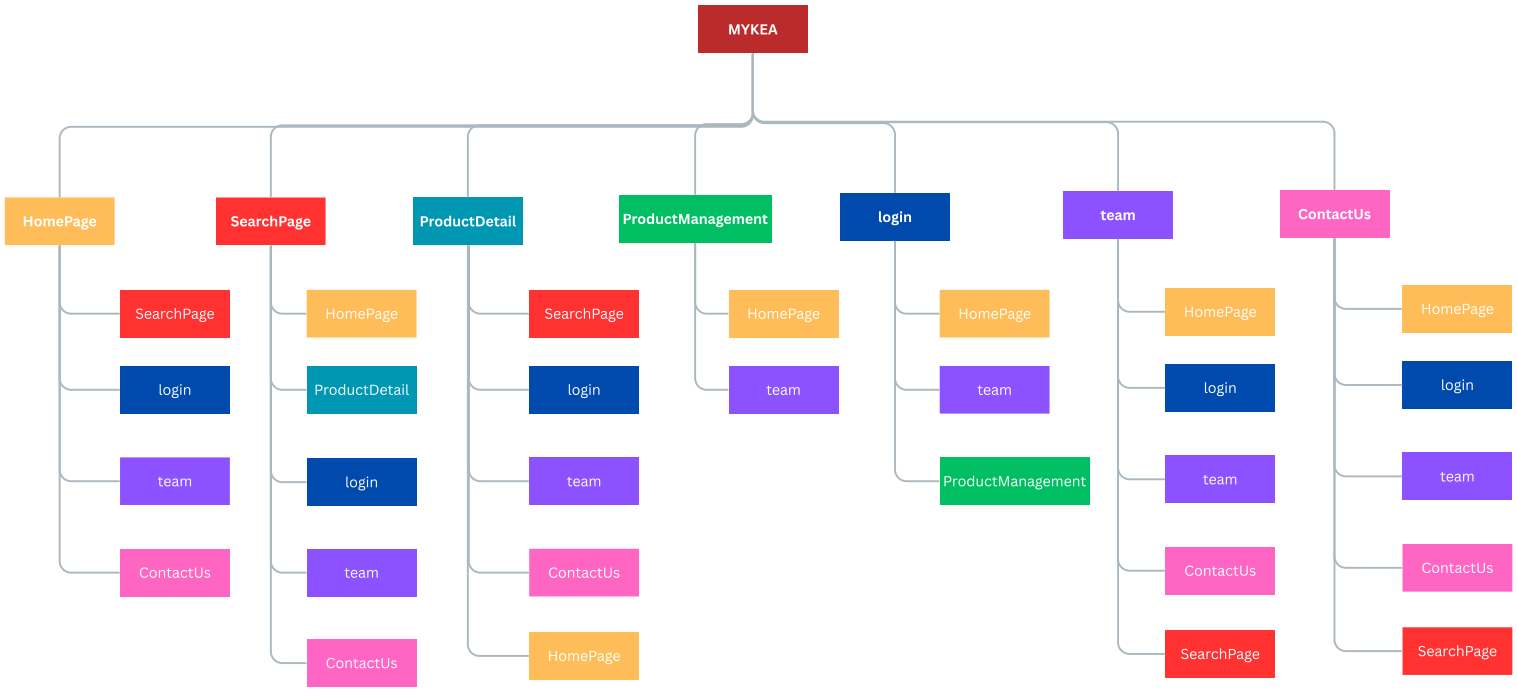


Data Dictionary

Table Name	Attribute Name	Description	Data Type	Data Format	Field Size	Key	FK Reference Table
Customer	CustomerID	Unique ID for customer	VARCHAR	XXXXX	10	PK	
	FName	Customer's first name	VARCHAR	XXXXX	50		
	LName	Customer's last name	VARCHAR	XXXXX	50		
	Email	Customer's email	VARCHAR	X@X.com	50		
	Password	Customer'sPassword	VARCHAR	XXXXX	255		
	Address	Shipping address	VARCHAR	XXXXX	255		
Product	ProductID	Unique product ID	VARCHAR	XXXXX	10	PK	
	PName	Product name	VARCHAR	XXXXX	100		
	PDescription	Product details	TEXT	XXXXX			
	Price	Product price	DECIMAL (10,2)	XXXXX.XX			
	Stock	Available stock quantity	INT	XXXXX	100		
	ProdCategoryID	Category ID	VARCHAR	X	100	FK	CategoryID [Category]
Stock	StockID	Unique stock ID	VARCHAR	XXXXX	10	PK	
	ProductID		VARCHAR	XXXXX	100	FK	ProductID [Product]

	Quantity		**INT**	**XXXXX**	**100**		
	Warehouse Location		**VARCHAR**	**XXXXX**	**100**		
Category	**CategoryID**	**Unique category ID**	**VARCHAR**	**X**	**100**	**PK**	
	CName	**Category name**	**VARCHAR**	**XXXXX**	**50**		
Orders	**OrderID**	**Unique order ID**	**VARCHAR**	**XXXXX**	**10**	**PK**	
	CustomerID	**Unique ID for customer**	**VARCHAR**	**XXXXX**	**10**	**FK**	**CustomerID [Customer]**
	OrderDate	**Date of order**	**DATE**	**YYYY-MM-DD**			
	TotalPrice	**Total cost**	**DECIMAL (10,2)**	**XXXXX.XX**			
OrderDetails	**UnitPrice**	**Product price per piece**	**DECIMAL (10,2)**	**XXXXX.XX**	**50**		
	OrderID	**Unique order ID**	**VARCHAR**	**XXXXX**	**50**	**FK**	**OrderID [Orders]**
	ProductID	**Unique product ID**	**VARCHAR**	**XXXXX**	**50**	**FK**	**ProductID [Product]**
	Quantity	**Quantity ordered**	**INT**	**XXXX**	**-**		

Navigation Diagram of web application



Detail of our Website Application and Website Service

1. Import crucial modules

```
const express = require('express');
const path = require('path');
const dotenv = require("dotenv");
```

- These codes import crucial modules that will make servers work like
- **express -** providing tools for routing, handling request and response.
- **path -** provides utilities for working with file and directory paths.
- **dotenv -** load configuration settings from .env file.

2. Setting Up express

```
app.use(router)
dotenv.config();

app.use('/front-end', express.static(path.join(__dirname, 'front-end')));

router.use(express.json());
router.use(express.urlencoded({ extended: true}));

app.listen(process.env.PORT, function () {

    console.log("Server listening at Port "
    + process.env.PORT);});
```

1. **app.use(router);**
   it will handle any incoming requests to the router object
2. **dotenv.config();**
   it executes the config function from the .env file

3. **app.use('/front-end', express.static(path.join(__dirname, 'front-end')));**
   It tells the server to serve static files (like HTML, CSS, JS, and images) from specific folders on the server, in this case is (‘front-end’).

4. **router.use(express.json());**
   router.use(express.urlencoded({ extended: true}));
   These two lines equip the router with the ability to automatically understand and process data sent in the *body* of incoming requests, whether that data is in JSON format or URL, this makes the data readily available in req.body.

5. **app.listen();**
   method in an Express.js application starts the server and makes it listen for incoming connections on a specified network port.

3. Setup/Connect to the database

```
const mysql = require('mysql2');
var connection = mysql.createConnection({
host     : process.env.DB_HOST,
user     : process.env.DB_USERNAME,
password : process.env.DB_PASSWORD,
database : process.env.DB_NAME
});
/* Connect to DB */
connection.connect(function(err){
   if(err) throw err;
   console.log(`Connected DB: ${process.env.DB_NAME}`);
});
```

- First, we import the MySQL modules, next we set up the details needed to connect to a MySQL database by using .env to keep

sensitive credentials like username and password out of the source code itself, and error handling the connection.

4. Setup Routing

```
//Login
router.get(['/Login'], (req, res) => {
   console.log("Request at " + req.url);
   res.sendFile(path.join(`${__dirname}/front-end/html/login.html`))
});

//HomePage
router.get(['/', '/HomePage'], (req, res) => {
    console.log("Request at " + req.url);
    res.sendFile(path.join(`${__dirname}/front-end/html/HomePage.html`))
});

//SearchPage
router.get('/SearchPage', (req, res) => {
    console.log("Request at " + req.url);
    res.sendFile(path.join(`${__dirname}/front-end/html/SearchPage.html`))
});

//ProductDetail
router.get('/ProductDetail', (req, res) => {
    console.log("Request at " + req.url);
    res.sendFile(path.join(`${__dirname}/front-end/html/ProductDetail.html`))
});

//ProductManagement
router.get('/ProductManagement', (req, res) => {
   console.log("Request at " + req.url);
   res.sendFile(path.join(`${__dirname}/front-end/html/ProductManagement.html`))
});

//OurTeam
router.get('/OurTeam', (req, res) => {
    console.log("Request at " + req.url);
    res.sendFile(path.join(`${__dirname}/front-end/html/Team.html`))
});

//Contact
router.get('/Contact-Us', (req, res) => {
```

```
    console.log("Request at " + req.url);
    res.sendFile(path.join(`${__dirname}/front-end/html/Contact-Us.html`))
});
```

- This code is about setting up the routing of our website such as if user go to (‘/’, ‘/Hompage’) it will send HomePage.html or if user go to (‘/SearchPage’) it will send SearchPage.html and log the req.url.

5. Route for handling login attempts

```
router.post('/api/login', (req, res) => { … });
```

- This code is a Api endpoint (‘/api/login’) to handle login attempts via HTTP post request
  1. It extracts the email and password sent in the request body.
  2. It performs basic validation to ensure both email and password are provided.
  3. It queries a database (user table) to find a user matching the provided email.
  4. It checks for the database error or if the user was not found.
  5. It directly compares the submitted plain text password with the plain text password (pwd) stored in the database.
  6. Checking whether the password match or not match, if match send back the 200 status and set the success to be true, if not match send back 401 status and set the success to be false.

6. Setup CRUD operations

1. Select all products

```
router.get('/product', function (req, res){
   connection.query(`SELECT * FROM product`, function (error,
results) {
      if(results.length ==0){
         return res.status(400).send({error: true, message: 'Your
database didnt have any data.'})
      }
      else{
         console.log(`${results.length} rows returned`);
         return res.send({error: false, data: results, message:
'Select All Products successfully.'});
      }
   });
});
```

-This is an Api endpoint (‘/product’) that handles HTTP GET requests. When client sends a GET request to (‘/product’), the code executes an SQL query to select all data from the product table in the database, if no products are found send back an error response, if products are found, sends back the data and JSON response.

2. Select product by product code or category name

```
router.get('/product/:ProductCodeOrCatName', function (req, res){
   let product_CodeOrCatName = req.params.ProductCodeOrCatName;
   console.log(`Searching for: ${product_CodeOrCatName}`);
   const query = `SELECT * FROM product WHERE product_code = ? OR
category3_code LIKE ?`;
      connection.query(query, [product_CodeOrCatName,
product_CodeOrCatName + '%'], function (error, results) {
         if(results.length == 0){
            return res.status(400).send({error: true, message:
'Please Provide product code or category name.'})
         }
         console.log(`${results.length} rows returned`);
            return res.send({error: false, data: results, message:
'Select Product successfully.'});
      });
});
```

-This is an Api endpoint that handles HTTP GET requests to (‘/product/’) followed by a specific value (‘/product/SomeCode’ , ‘/product/SomeCategory’) designed to Select product by specific value. First it take the value provided in the URL after (‘/product/’) and it perform database query then query search for rows where either ‘product_code’ column or the ‘category3_code’ column and handling errors then send back the matching products and the JSON response back.

3. Insert product

```
router.post('/product', function (req, res){
    let product = req.body.product;
    console.log(product);
    if(!product){
       return res.status(400).send({error: true, message: 'Please
Provide product information'})
    }

    connection.query(`INSERT into product SET ?`, [product],
function (error, results) {
       if(error){
         throw error;
       }
       console.log(`New Product has been inserted`);
       return res.send({error: false, data: results.affectedRows,
message: 'Product has been inserted successfully.'});
    });

 });
```

- This is an Api endpoint that handles HTTP POST requests to (‘/product/’), designed to add a new product to the database. First it expects the client to send product data within request body then extract this product object from the request body, next validate the product , and execute SQL insert query to add new row to the product table, then handling errors and send back JSON response.

4. Update product

```
router.put('/product', function (req,res){
    let PreviousProduct_code = req.body.product.product_code;
    let NewProduct = req.body.newProduct;

    if (!PreviousProduct_code || !NewProduct){
       return res.status(400).send({error: true, message: 'Please
Provide Product information'})
    }

    connection.query(`UPDATE product SET ? WHERE product_code = ?`,
[NewProduct, PreviousProduct_code], function (error, results) {
       if(error){
         throw error;
       }
       console.log(`Product has been updated`);
       return res.send({error: false, data: results.affectedRows,
message: `Product has been updated successfully.`});
    });

 });
```

- This is an Api endpoint that handles HTTP PUT requests to (‘/product/’), designed to update the existing product. First it get the product code of the product that want to update then get the new information for new product then handling error, next execute SQL update query then handling error, if success send back the JSON response.

5. Delete product

```
router.delete('/product', function (req,res){
    let product_code = req.body.product.product_code;

    if (!product_code){
       return res.status(400).send({error: true, message: 'Please
provide product_code'})
    }

    connection.query(`DELETE FROM product WHERE product_code = ?`,
[product_code], function (error, results) {
       if(error){
         throw error;
       }
       console.log(`The product code ${product_code} has been
deleted.`);
       return res.send({error: false, data: results.affectedRows,
message: `The Product id ${product_code} has been deleted
successfully.`});
    });
 });
```

- This is an Api endpoint that handles HTTP DELETE requests to (‘/product/’), designed to delete a product from the database. First it gets the product code of the product that want to delete then handling error, next execute SQL delete query then handling error, if success send back the JSON response.

7. Client-Side

Styling & interaction (Boostrap, swiper)

- We use Boostrap for easy styling, bootstrap provides a lot of complete structures like card, nav, etc. so it can shorten working time,
  and for swiper to make an interaction sliding image and have the next and previous button for home page.

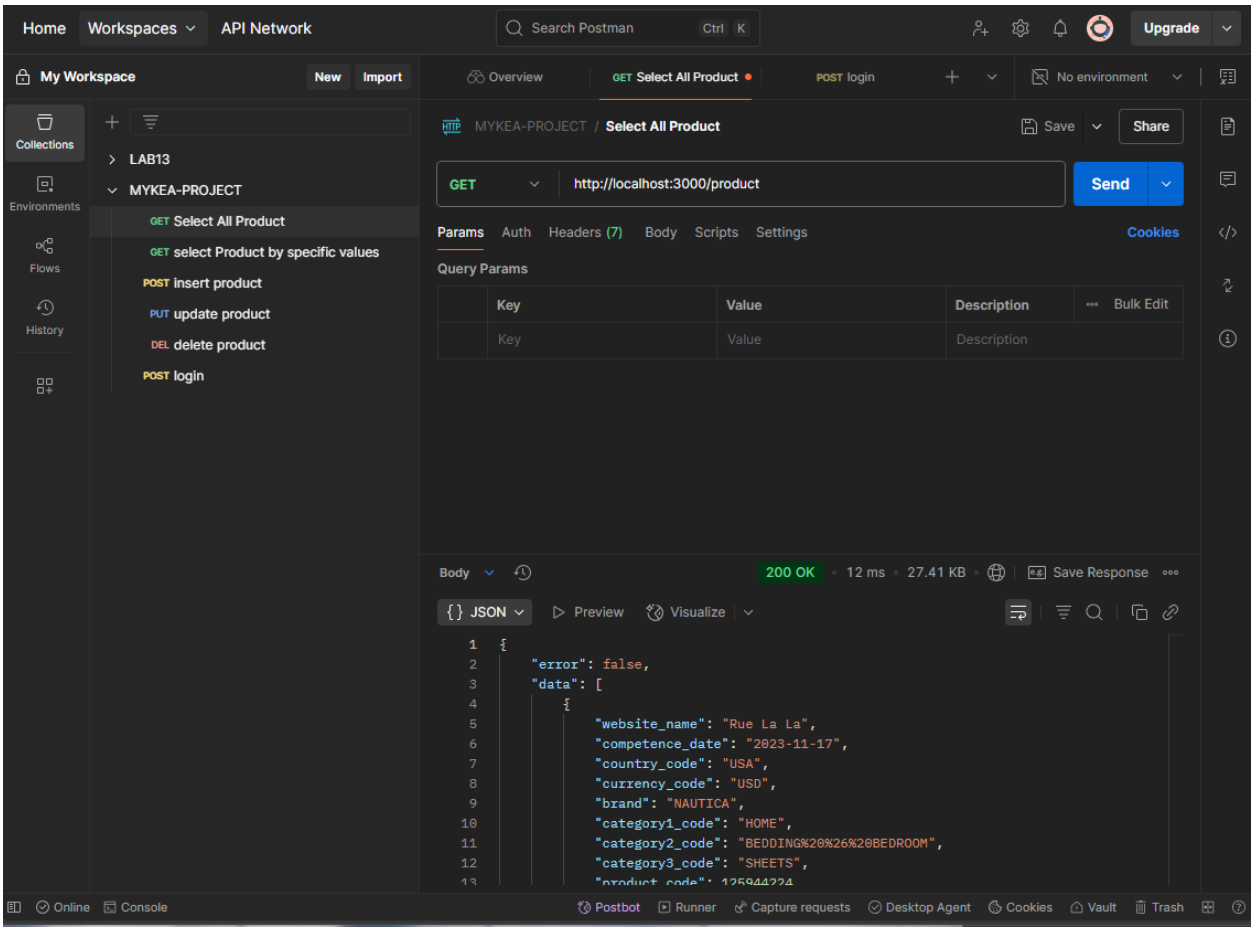
8. Public API

-our website use Longdo map, it an online web mapping service and platform that primarily focused on Thai, it provides many features such as zoom bar and the Pin. We use it to display our company location in Contact us page.

- **Link: Longdo Map API**

The testing results of web services using Postman or any program for testing web service

1. Select all products

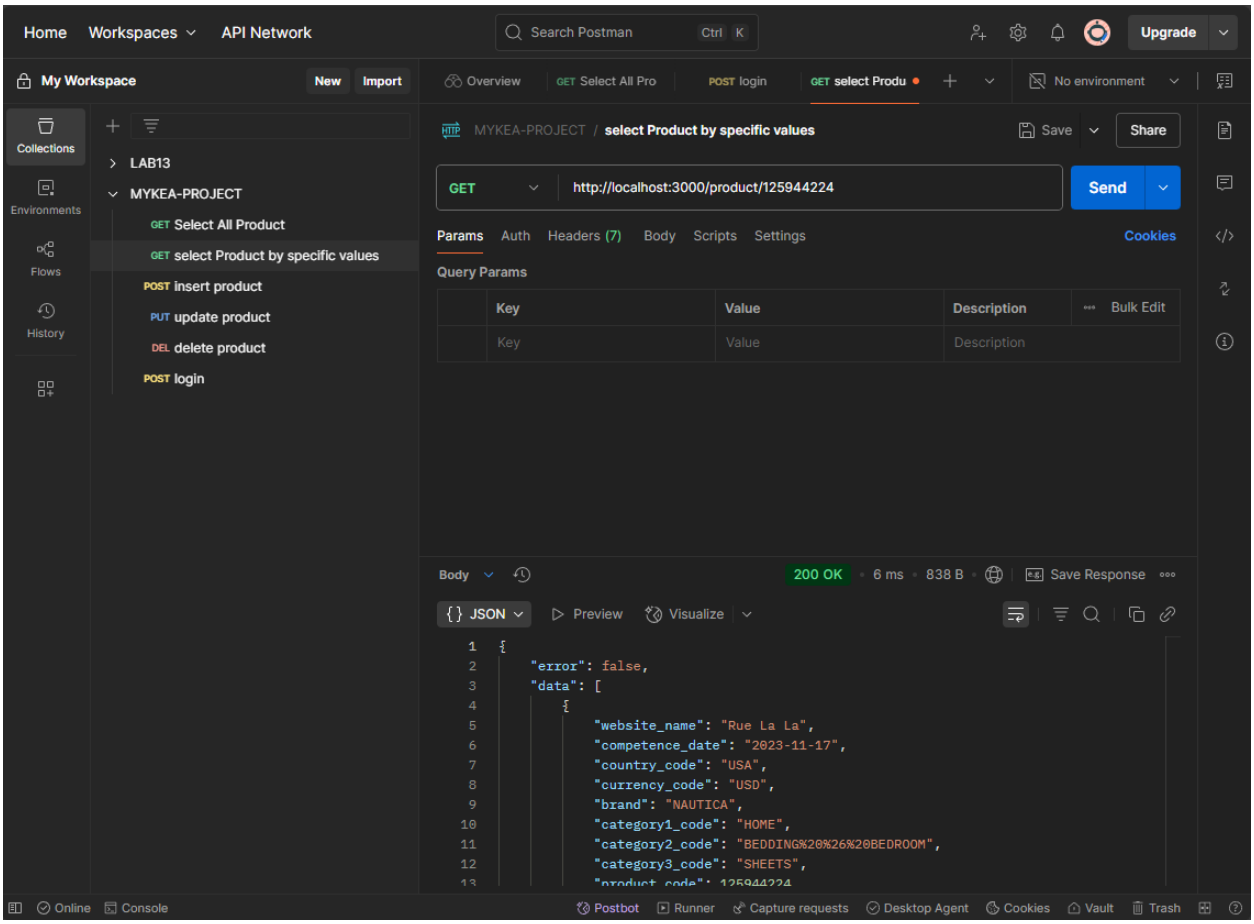


//Testing Select all products
//method: GET
//URL: http://localhost:3000/product
//body: raw JSON


```
{
   "error": false,
   "data": [
      {
         "website_name": "Rue La La",
         "competence_date": "2023-11-17",
         "country_code": "USA",
         "currency_code": "USD",
         "brand": "NAUTICA",
         "category1_code": "HOME",
         "category2_code": "BEDDING%20%26%20BEDROOM",
         "category3_code": "SHEETS",
         "product_code": 125944224,
         "title": "Nautica T200 Solid 4Pc Grey Sheet Set",
         "itemurl":
"https://www.ruelala.com/boutique/product/159137/125944224/",
         "imageurl":
"http://asset1.ruecdn.com/images/product/303394/3033948560_RLLDTH_1.jpg"
,
         "full_price": 80,
         "price": 39,
         "full_price_eur": 73.8,
         "price_eur": 35.97,
         "flg_discount": "1"
      }, … (other 49 results),

      "message": "Select All Products successfully."
}
```

2. Select product by specific values

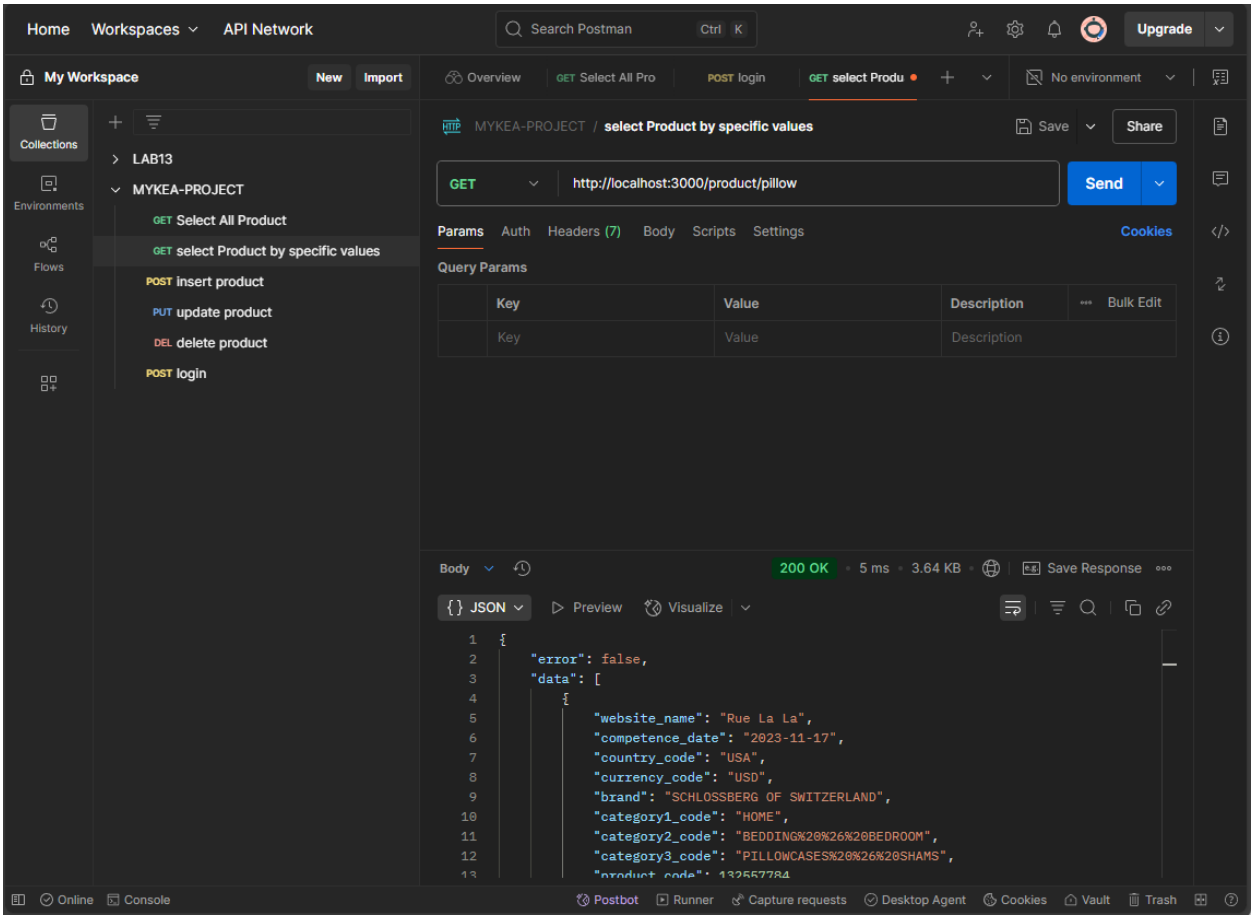
Test Case 1 (product_code):



//Testing Select product by specific values (product_code)
//method: GET
//URL: http://localhost:3000/product/125944224
//body: raw JSON


```
{
    "error": false,
    "data": [
        {
            "website_name": "Rue La La",
            "competence_date": "2023-11-17",
            "country_code": "USA",
            "currency_code": "USD",
            "brand": "NAUTICA",
            "category1_code": "HOME",
            "category2_code": "BEDDING%20%26%20BEDROOM",
            "category3_code": "SHEETS",
            "product_code": 125944224,
            "title": "Nautica T200 Solid 4Pc Grey Sheet Set",
            "itemurl":
"https://www.ruelala.com/boutique/product/159137/125944224/",
            "imageurl":
"http://asset1.ruecdn.com/images/product/303394/3033948560_RLLDTH_1.jpg"
,
            "full_price": 80,
            "price": 39,
            "full_price_eur": 73.8,
            "price_eur": 35.97,
            "flg_discount": "1"
        }
    ],
    "message": "Select Product successfully."
}
```

Test Case 2 (Category name):

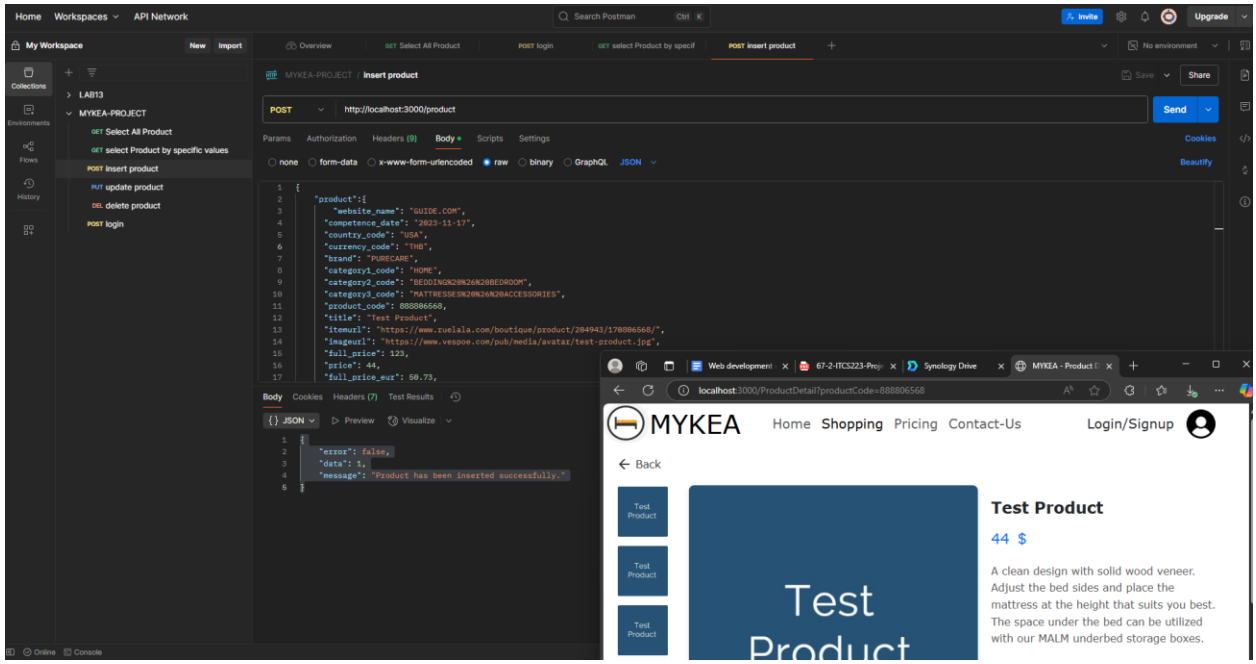


//Testing Select product by specific values (category3_code)
//method: GET
//URL: http://localhost:3000/product/pillow
//body: raw JSON
(I decrease the data inside “data” to decrease the amount of text for you to easily see, i will show what important to check (run this before insert, update, delete anything to be the same result.))


```
{
   "error": false,
   "data": [
      {
         "category3_code": "PILLOWCASES%20%26%20SHAMS",
         "product_code": 132557784,
      },

       . . . ,

       {
         "category3_code": "PILLOW%20INSERTS",
         "product_code": 180650918,
      }
   ],
   "message": "Select Product successfully."
}
```

3. Insert product

Test Case 1:



//Testing insert product
//method: POST
//URL: http://localhost:3000/product
//body: raw JSON

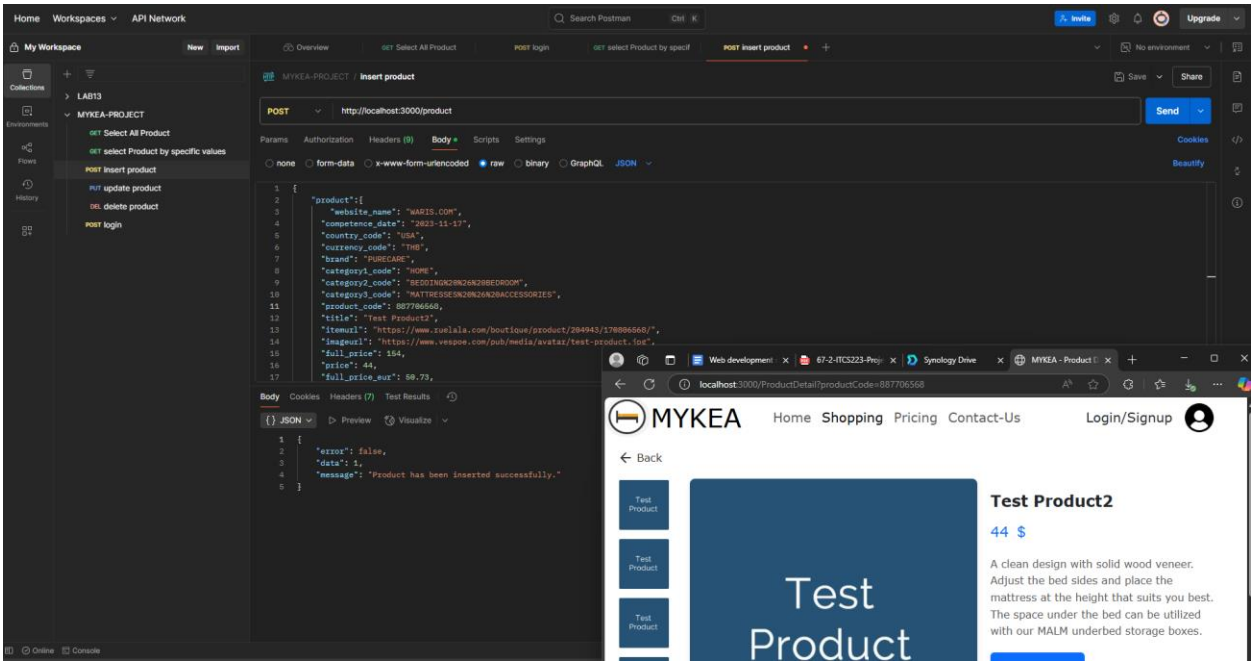
Client sent this:


```
{
   "product":{
      "website_name": "GUIDE.COM",
      "competence_date": "2023-11-17",
      "country_code": "USA",
      "currency_code": "THB",
      "brand": "PURECARE",
      "category1_code": "HOME",
      "category2_code": "BEDDING%20%26%20BEDROOM",
      "category3_code":
"MATTRESSES%20%26%20ACCESSORIES",
      "product_code": 888806568,
      "title": "Test Product",
      "itemurl":
"https://www.ruelala.com/boutique/product/204943/170806568/",
      "imageurl": "https://www.vespoe.com/pub/media/avatar/test-
product.jpg",
      "full_price": 123,
      "price": 44,
      "full_price_eur": 50.73,
      "price_eur": 40.59,
      "flg_discount": "1"
   }
}
```


Response:


```
{
   "error": false,
   "data": 1,
   "message": "Product has been inserted successfully."
}
```

Test Case 2:


Home
Workspaces
API Network
Search Postman
Invite
Upgrade
My Workspace
New
Import
Overview
GET Select All Product
POST login
GET select Product by specif
POST insert product
No environment
Collections
Environments
Flows
History
LAB13
MYKEA-PROJECT
GET Select All Product
GET select Product by specific values
POST Insert product
PUT update product
DEL delete product
POST login
MYKEA-PROJECT / insert product
Save
Share
POST
http://localhost:3000/product
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"product":[
"website_name": "WARIS.COM",
"competence_date": "2023-11-17",
"country_code": "USA",
"currency_code": "TWD",
"brand": "PUBECARE",
"category1_code": "HOME",
"category2_code": "BEDDINGN20N26N20BEDROOM",
"category3_code": "MATTRESSES%20%26%20ACCESSORIES",
"product_code": 887706568,
"title": "Test Product2",
"itemurl": "https://www.zuelala.com/boutique/product/204943/178806568/",
"imageurl": "https://www.vespoe.com/pub/media/avatar/test-product.jpg",
"full_price": 154,
"price": 44,
"full_price_eur": 50.73,
Body
Cookies
Headers (7)
Test Results
{} JSON
Preview
Visualize
{
"error": false,
"data": 1,
"message": "Product has been inserted successfully."
}
Online
Console
Web development
67-2-ITCS223-Proj
Synology Drive
MYKEA - Product D
localhost:3000/ProductDetail?productCode=887706568
MYKEA
Home
Shopping
Pricing
Contact-Us
Login/Signup
Back
Test Product
Test Product
Test Product
Test Product
Test Product2
44 $
A clean design with solid wood veneer. Adjust the bed sides and place the mattress at the height that suits you best. The space under the bed can be utilized with our MALM underbed storage boxes.

//Testing insert product
//method: POST
//URL: http://localhost:3000/product
//body: raw JSON

Client:

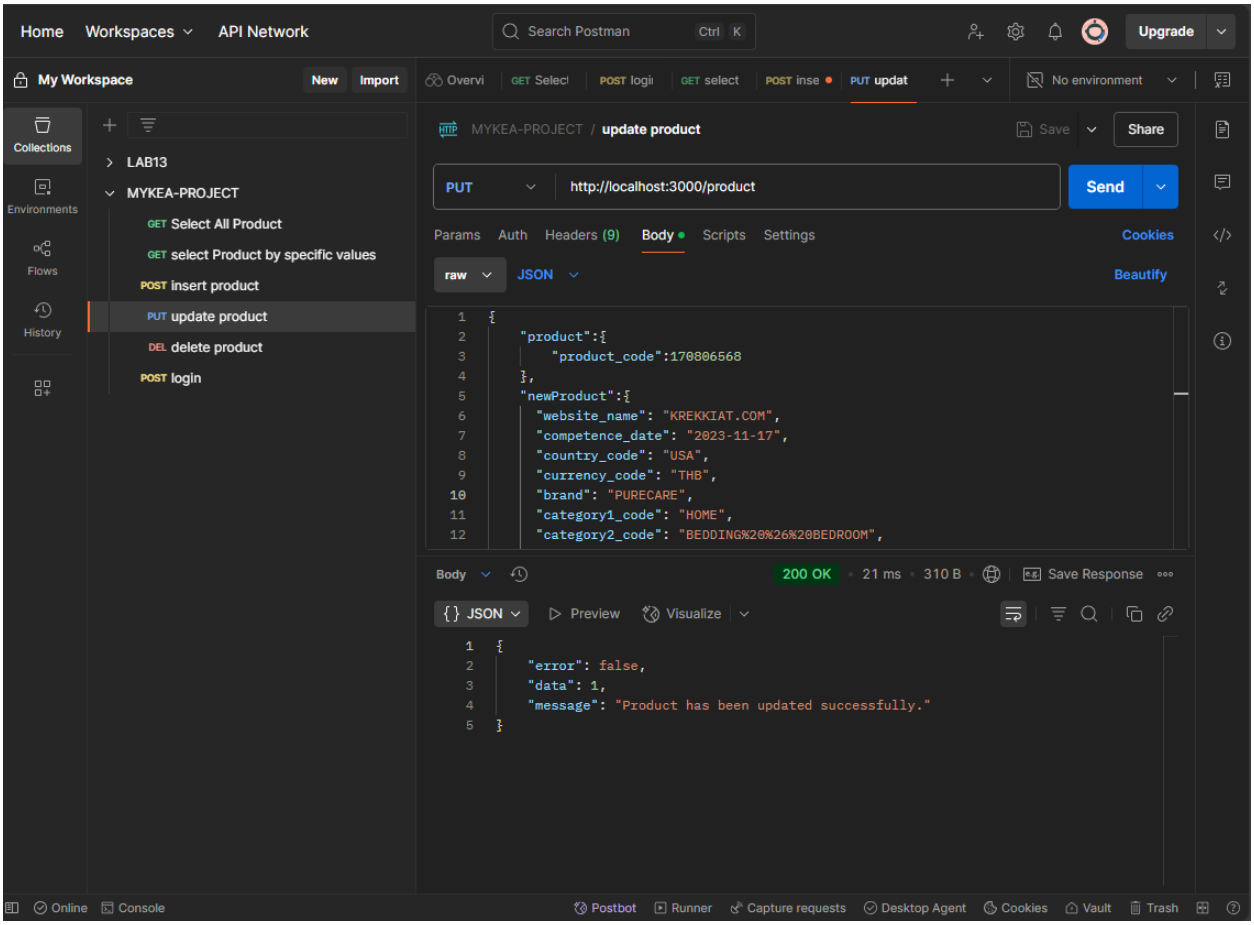

```
{
   "product":{
      "website_name": "WARIS.COM",
     "competence_date": "2023-11-17",
     "country_code": "USA",
     "currency_code": "THB",
     "brand": "PURECARE",
     "category1_code": "HOME",
     "category2_code": "BEDDING%20%26%20BEDROOM",
     "category3_code": "MATTRESSES%20%26%20ACCESSORIES",
     "product_code": 887706568,
     "title": "Test Product2",
     "itemurl": "https://www.ruelala.com/boutique/product/204943/170806568/",
     "imageurl": "https://www.vespoe.com/pub/media/avatar/test-product.jpg",
     "full_price": 154,
     "price": 44,
     "full_price_eur": 50.73,
     "price_eur": 40.59,
     "flg_discount": "1"
   }
}
```


Response:


```
{
   "error": false,
   "data": 1,
   "message": "Product has been inserted successfully."
}
```

4. Update product

Test Case 1:



SQL:

KREKKIAT.COM	2023-11-17	USA	THB	PURECARE	HOME	BEDDING

//Testing update product
//method: PUT
//URL: http://localhost:3000/product
//body: raw JSON

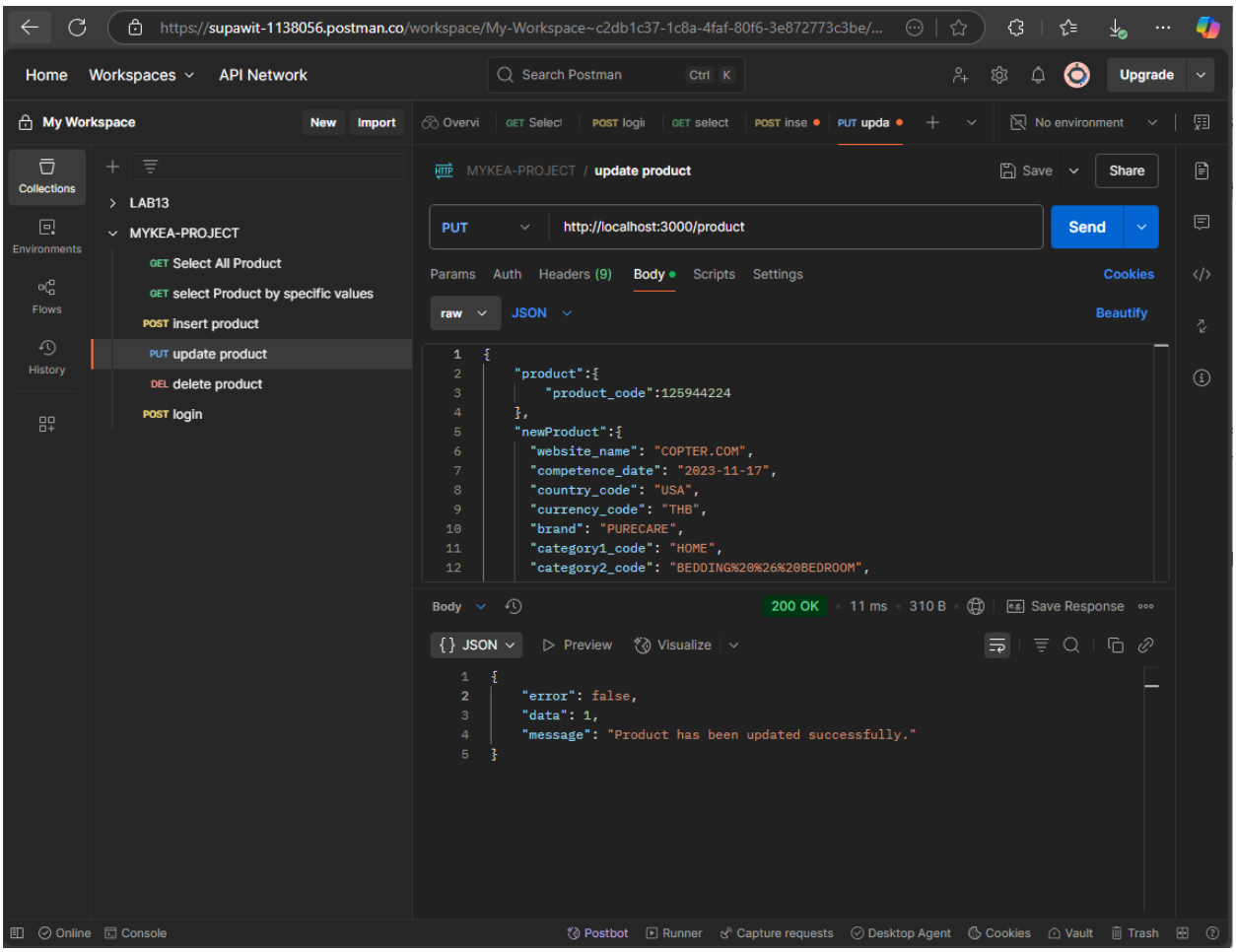
Client:


```
{
   "product":{
      "product_code":170806568
   },
   "newProduct":{
     "website_name": "KREKKIAT.COM",
     "competence_date": "2023-11-17",
     "country_code": "USA",
     "currency_code": "THB",
     "brand": "PURECARE",
     "category1_code": "HOME",
     "category2_code": "BEDDING%20%26%20BEDROOM",
     "category3_code": "MATTRESSES%20%26%20ACCESSORIES",
     "title": "PureCare Cotton Terry Mattress Protector",
     "itemurl": "https://www.ruelala.com/boutique/product/204943/170806568/",
     "imageurl":
"http://asset2.ruecdn.com/images/product/303091/3030916670_RLLDTH_1.jpg"
,
     "full_price": 55,
     "price": 44,
     "full_price_eur": 50.73,
     "price_eur": 40.59,
     "flg_discount": "1"
   }

}
```


Response:


```
{
   "error": false,
   "data": 1,
   "message": "Product has been updated successfully."
}
```

Test Case 2:



SQL:

COPTER.COM	2023-11-17	USA	THB	PURECARE	HOME	BEDDING

//Testing update product
//method: PUT
//URL: http://localhost:3000/product
//body: raw JSON

Client:

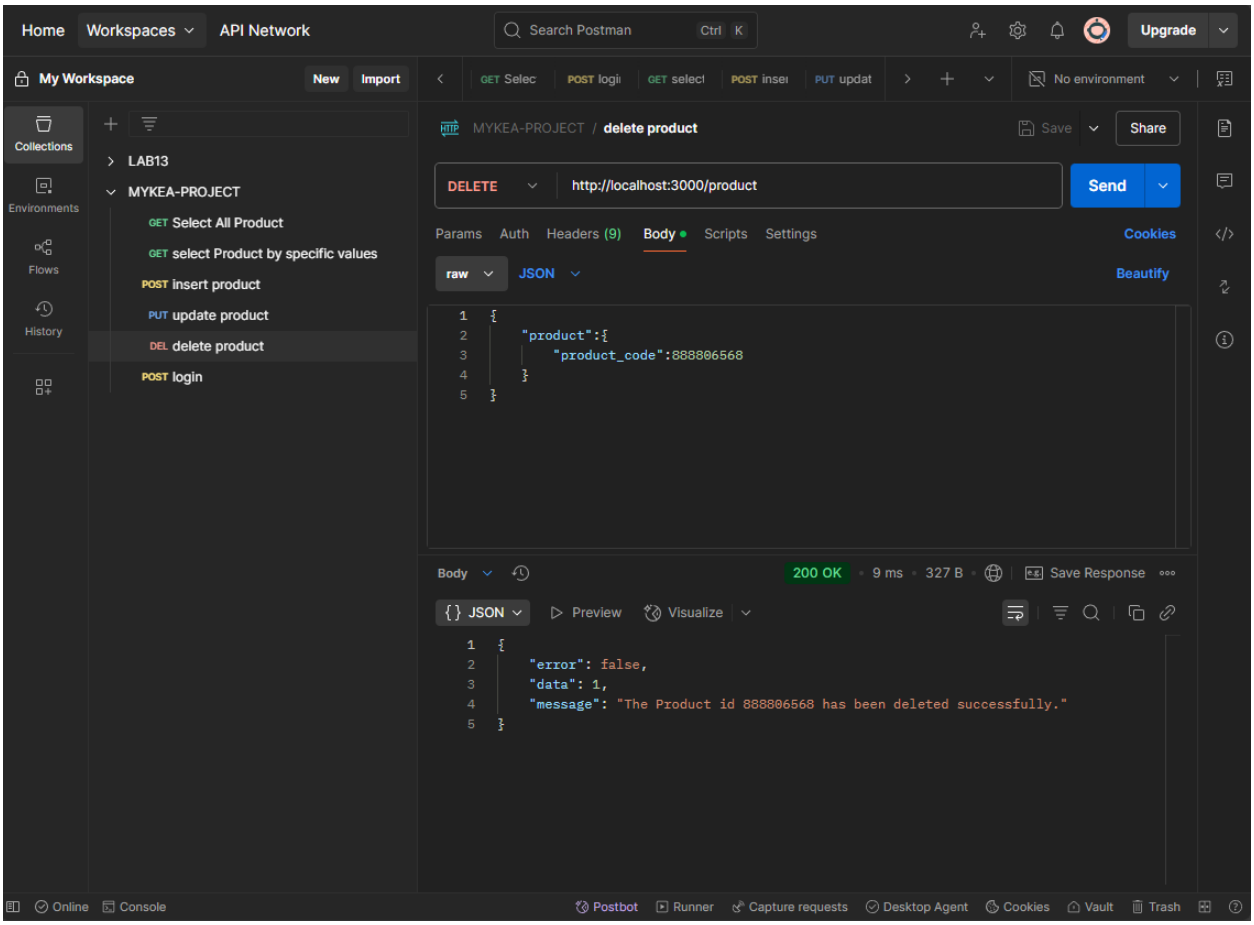

```
{
   "product":{
      "product_code":125944224
   },
   "newProduct":{
     "website_name": "COPTER.COM",
     "competence_date": "2023-11-17",
     "country_code": "USA",
     "currency_code": "THB",
     "brand": "PURECARE",
     "category1_code": "HOME",
     "category2_code": "BEDDING%20%26%20BEDROOM",
     "category3_code": "MATTRESSES%20%26%20ACCESSORIES",
     "title": "PureCare Cotton Terry Mattress Protector",
     "itemurl": "https://www.ruelala.com/boutique/product/204943/170806568/",
     "imageurl":
"http://asset2.ruecdn.com/images/product/303091/3030916670_RLLDTH_1.jpg"
,
     "full_price": 55,
     "price": 44,
     "full_price_eur": 50.73,
     "price_eur": 40.59,
     "flg_discount": "1"
   }

}
```


Response:


```
{
   "error": false,
   "data": 1,
   "message": "Product has been updated successfully."
}
```

5. Delete product

Test Case 1:



//Testing delete product
//method: DELETE
//URL: http://localhost:3000/product
//body: raw JSON

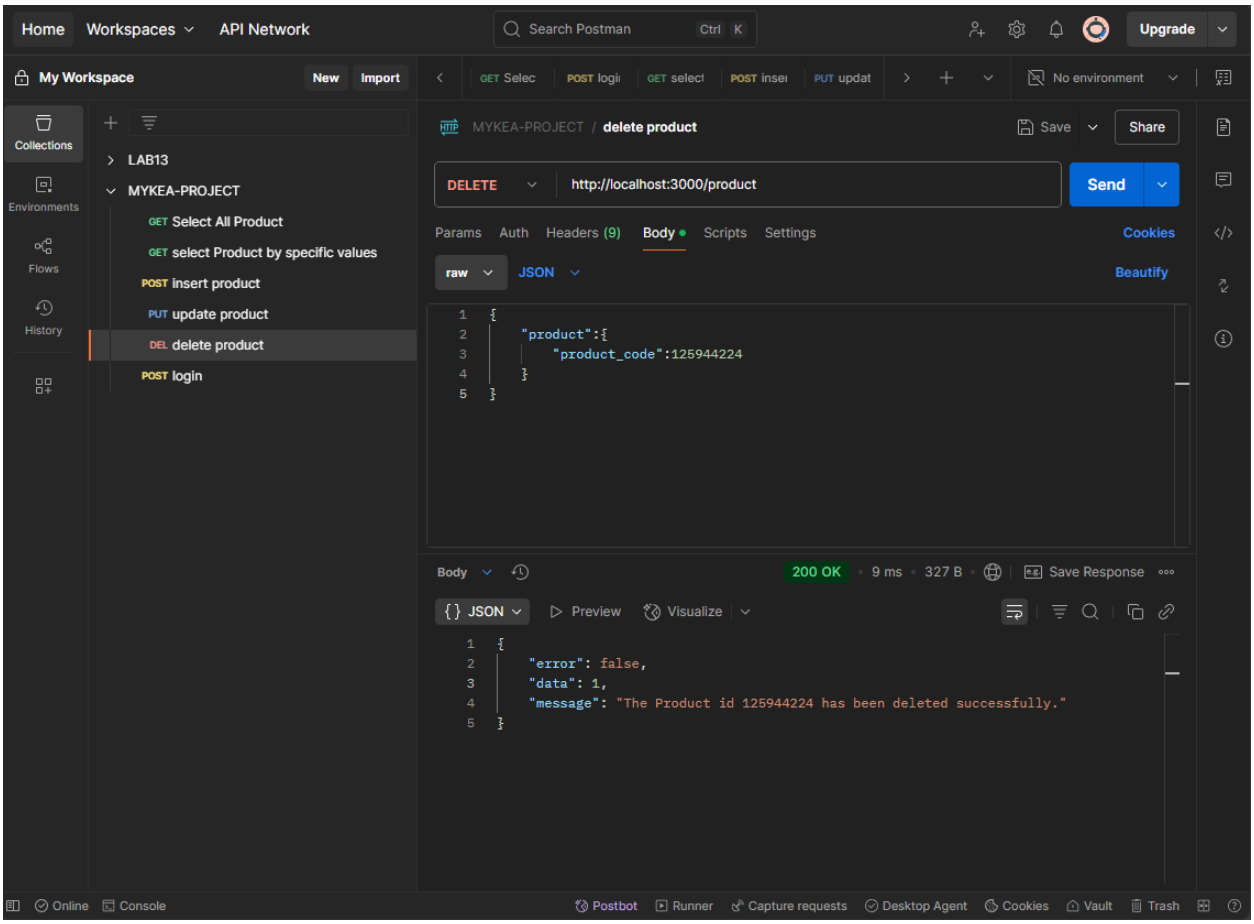
Client:


```
{
   "product":{
      "product_code":888806568
   }
}
```


Response:


```
{
   "error": false,
   "data": 1,
   "message": "The Product id 888806568 has been deleted successfully."
}
```

Test Case 2:


Home
Workspaces
API Network
Search Postman
Upgrade
My Workspace
New
Import
No environment
Collections
Environments
Flows
History
LAB13
MYKEA-PROJECT
GET Select All Product
GET select Product by specific values
POST insert product
PUT update product
DEL delete product
POST login
MYKEA-PROJECT / delete product
Save
Share
DELETE
http://localhost:3000/product
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
raw
JSON
Beautify
{
"product":{
"product_code":125944224
}
}
Body
200 OK
9 ms
327 B
Save Response
JSON
Preview
Visualize
{
"error": false,
"data": 1,
"message": "The Product id 125944224 has been deleted successfully."
}
Online
Console
Postbot
Runner
Capture requests
Desktop Agent
Cookies
Vault
Trash

//Testing delete product
//method: DELETE
//URL: http://localhost:3000/product
//body: raw JSON

Client:

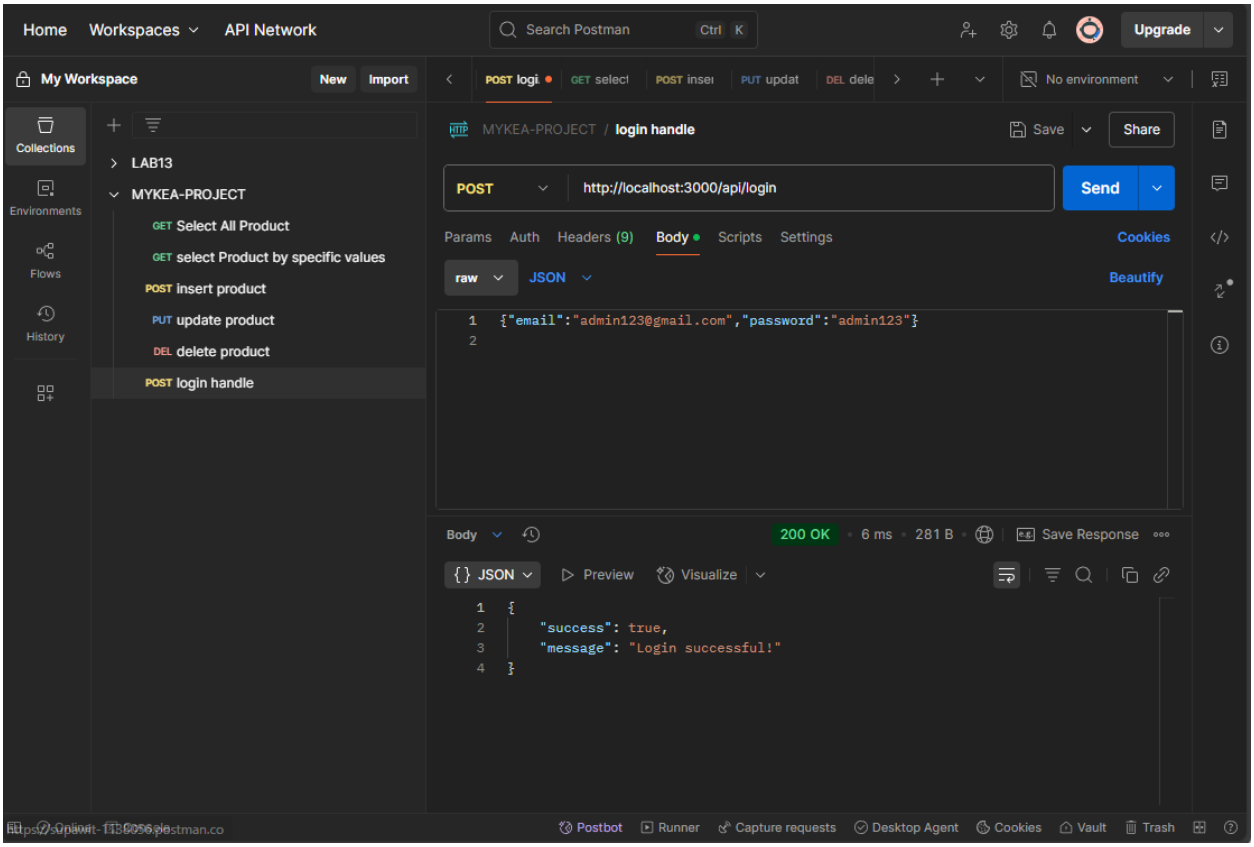

```
{
   "product":{
      "product_code":125944224
   }
}
```

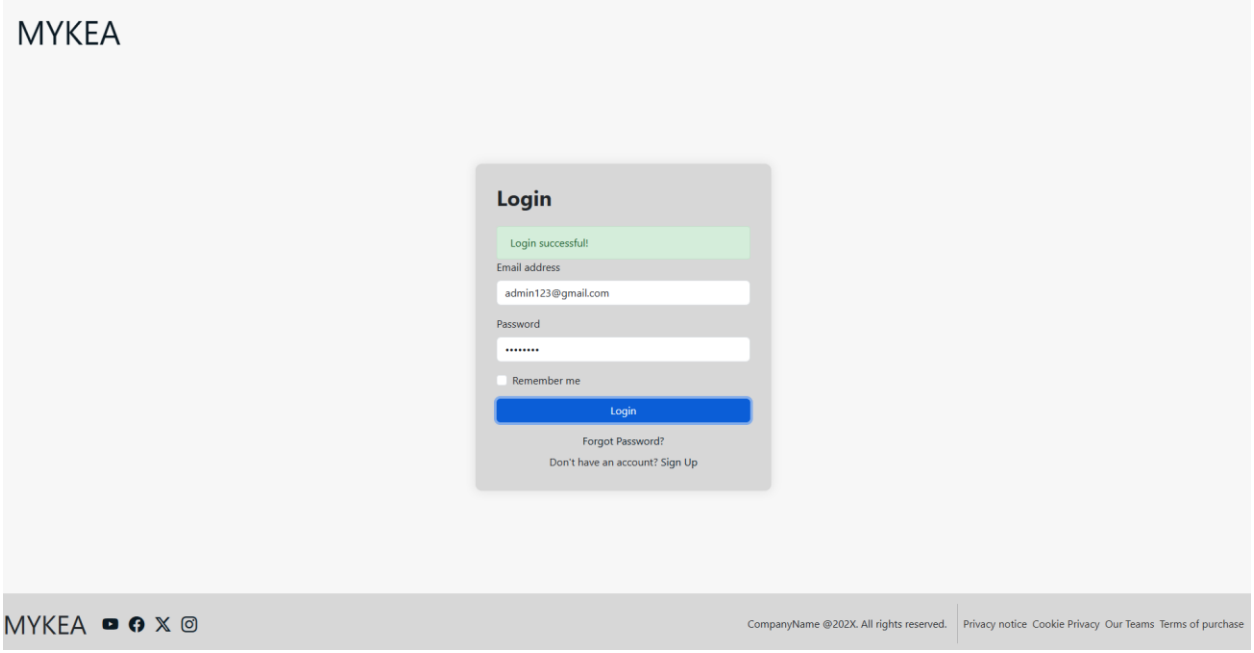

Response:


```
{
   "error": false,
   "data": 1,
   "message": "The Product id 125944224 has been deleted successfully."
}
```

6. Login handle

Test Case 1:



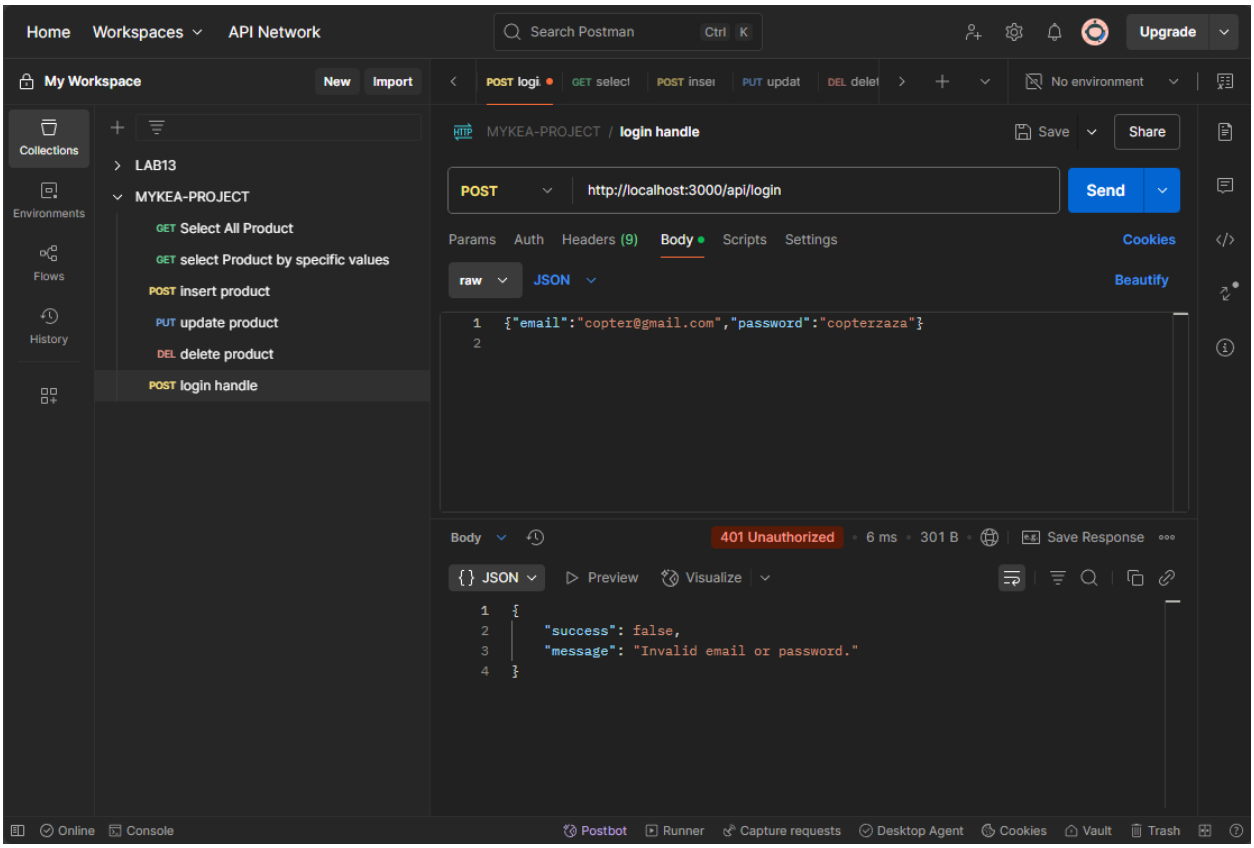


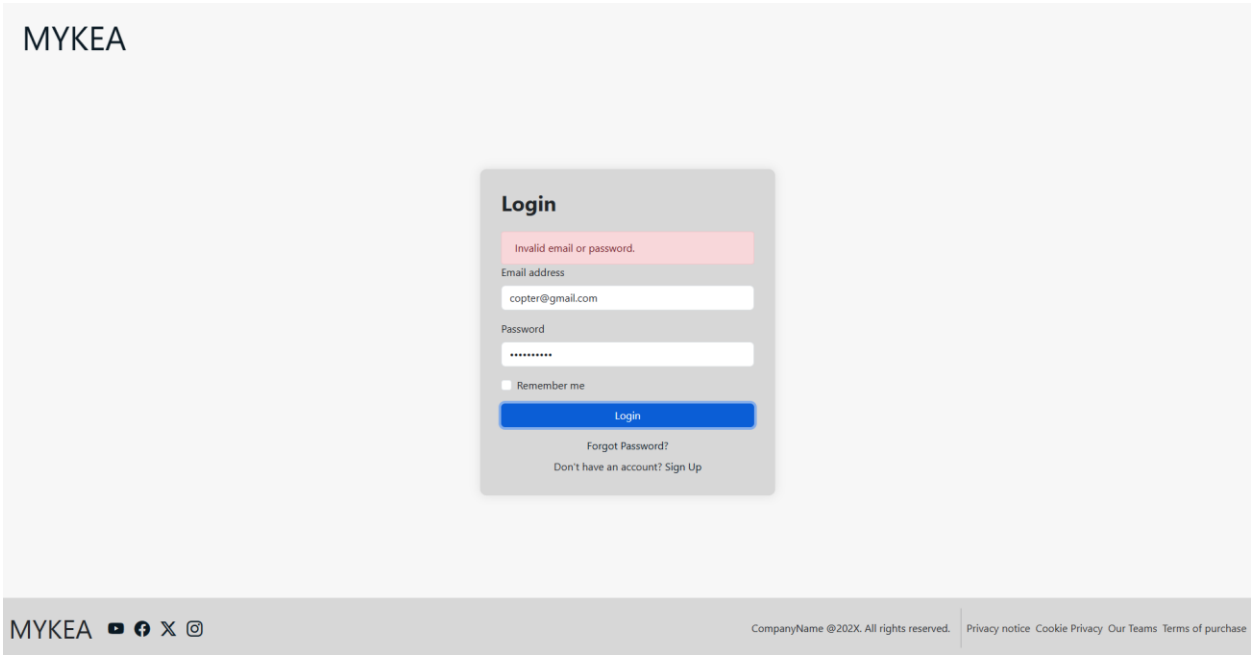
//Testing login handle
//method: POST
//URL: http://localhost:3000/api/login
//body: raw JSON

Client:
{"email":"admin123@gmail.com","password":"admin123"}

Response:
{
    "success": true,
    "message": "Login successful!"
}

Test Case 2:


Home
Workspaces
API Network
Search Postman
Upgrade
My Workspace
New
Import
No environment
Collections
Environments
Flows
History
LAB13
MYKEA-PROJECT
GET Select All Product
GET select Product by specific values
POST insert product
PUT update product
DEL delete product
POST login handle
MYKEA-PROJECT / login handle
Save
Share
POST
http://localhost:3000/api/login
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
raw
JSON
Beautify
{"email":"copter@gmail.com","password":"copterzaza"}
Body
401 Unauthorized
6 ms
301 B
Save Response
JSON
Preview
Visualize
{
"success": false,
"message": "Invalid email or password."
}
Online
Console
Postbot
Runner
Capture requests
Desktop Agent
Cookies
Vault
Trash


MYKEA
Login
Invalid email or password.
Email address
copter@gmail.com
Password
Remember me
Login
Forgot Password?
Don't have an account? Sign Up
MYKEA
CompanyName ©202X. All rights reserved.
Privacy notice Cookie Privacy Our Teams Terms of purchase

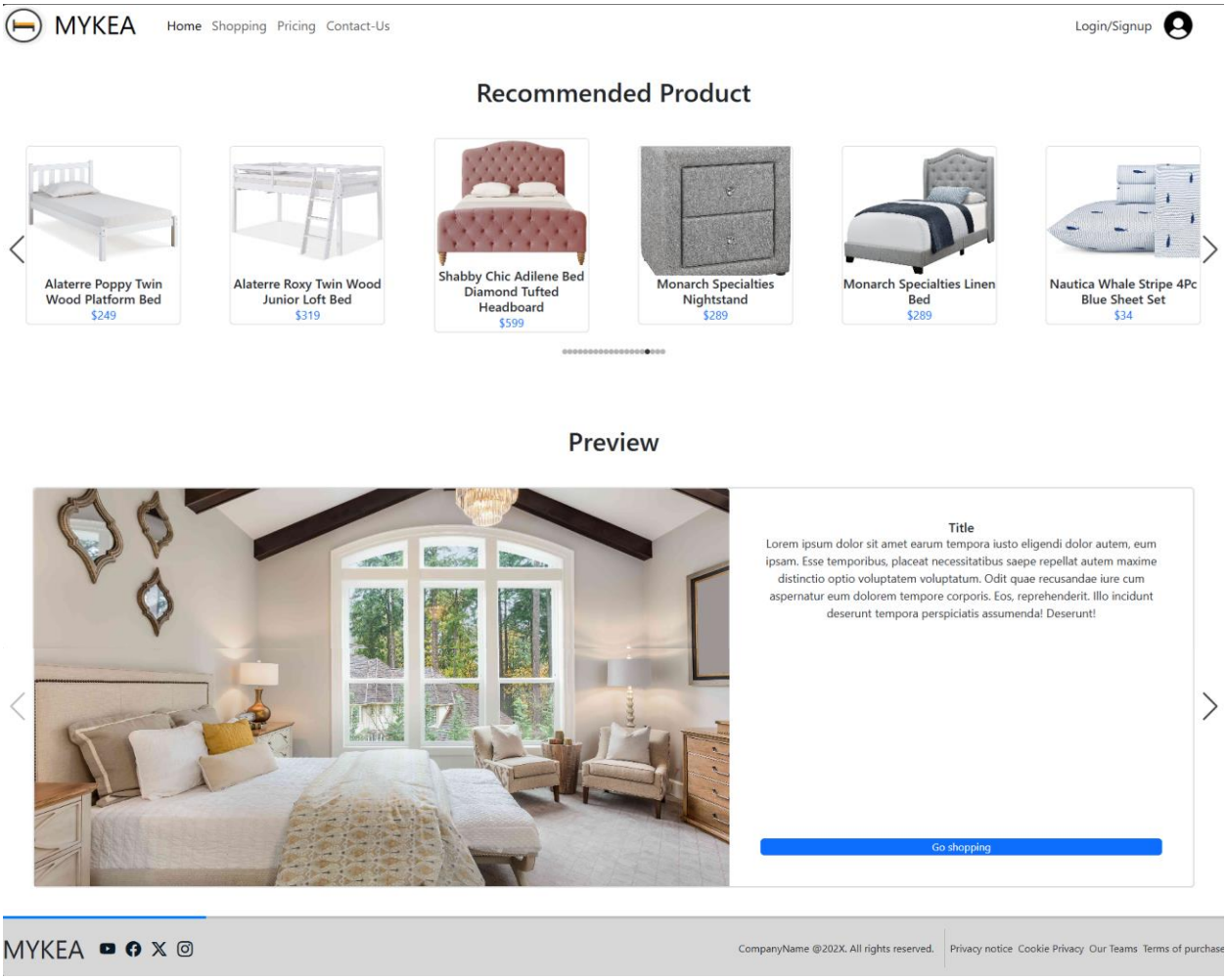
//Testing login handle
//method: POST
//URL: http://localhost:3000/api/login
//body: raw JSON

Client:
{"email":"copter@gmail.com","password":"copterzaza"}

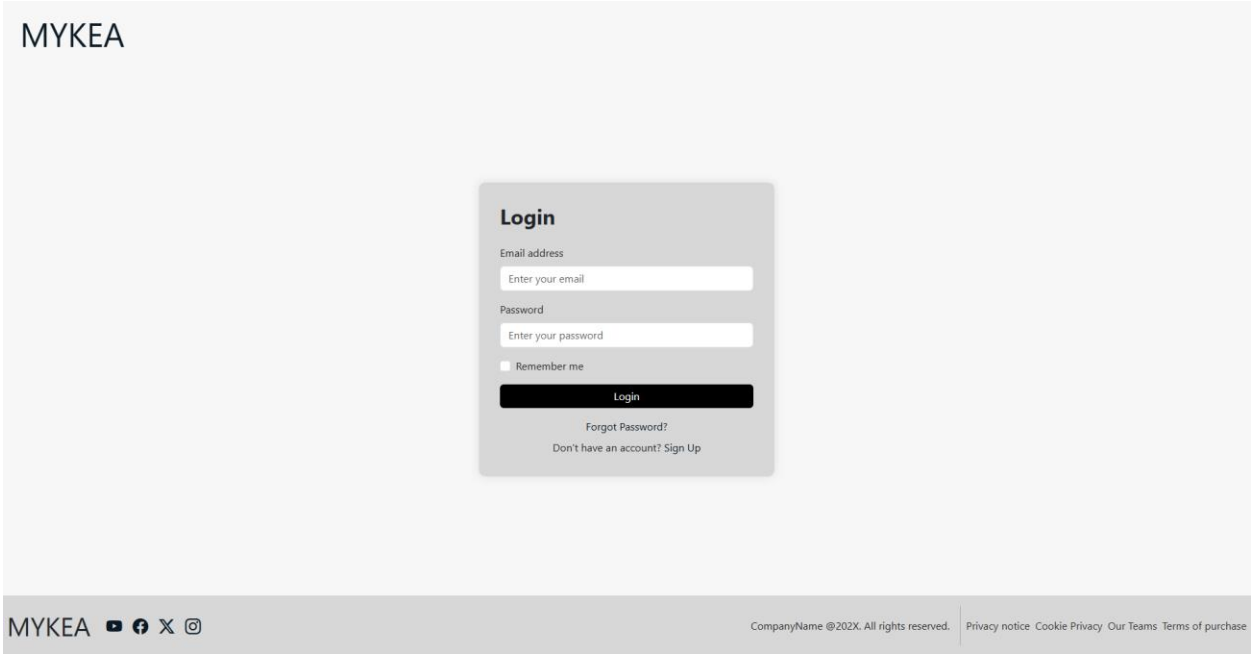
Response:
{
"success": false,
"message": "Invalid email or password."
}

Example of web application pages

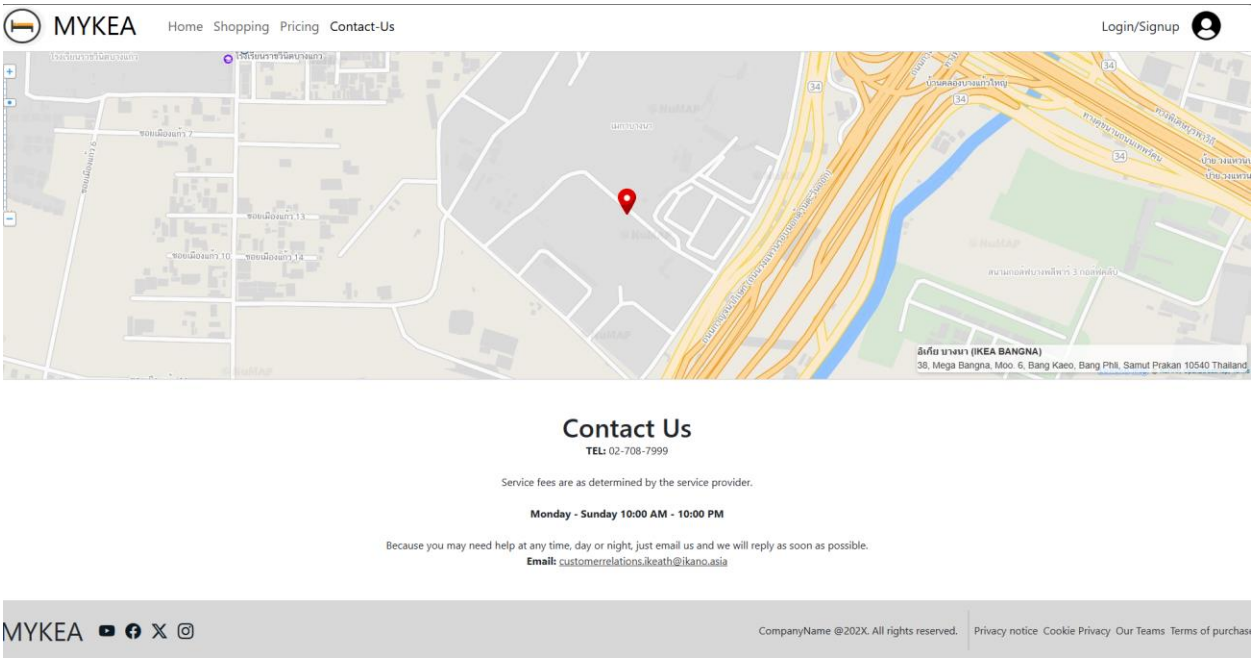
- Home page (‘http://localhost:3000/’, ‘http://localhost:3000/HomePage’)



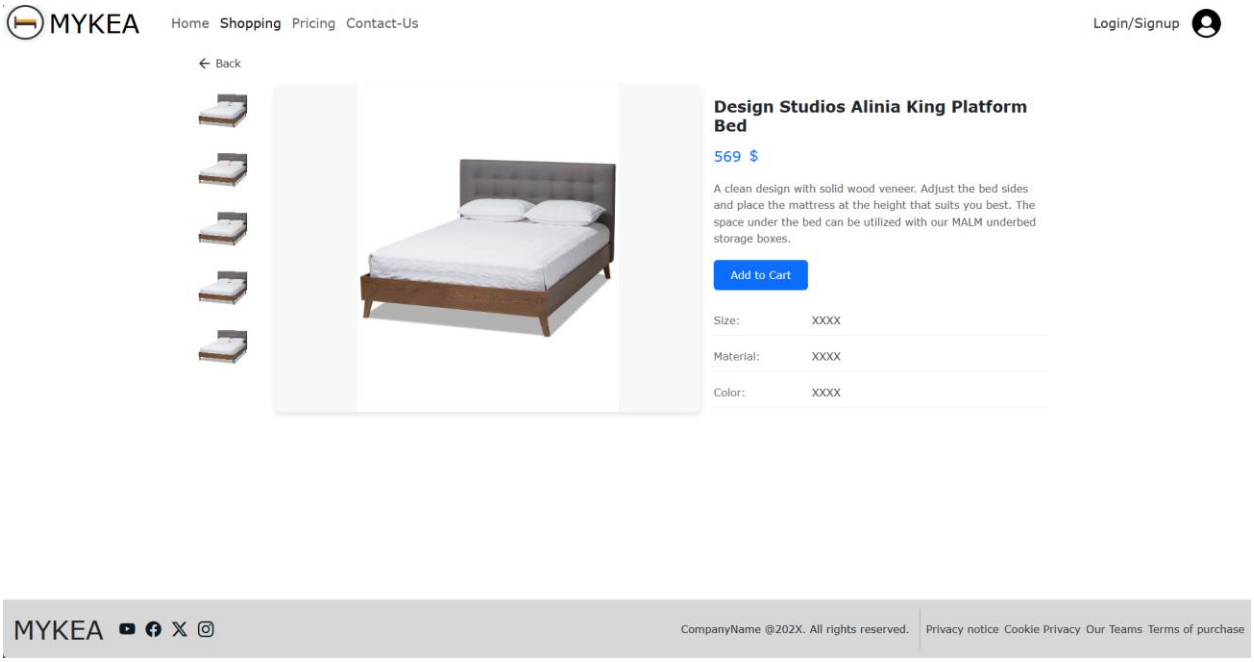
- Login page (‘http://localhost:3000/Login’)



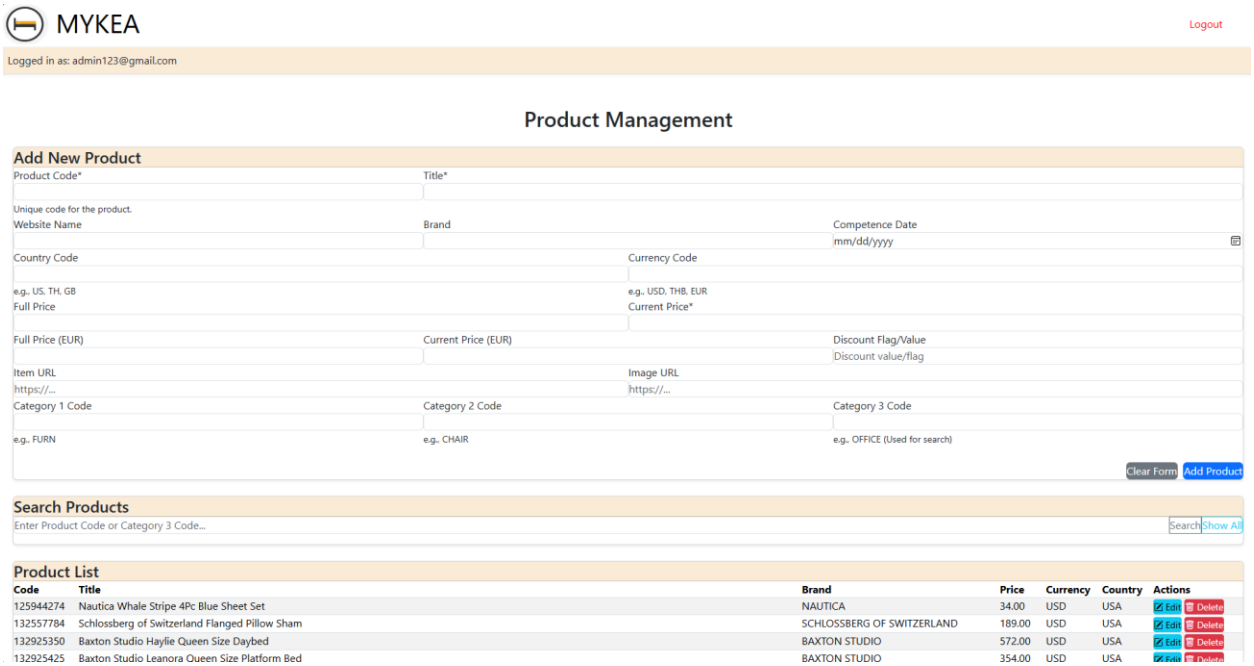
- Contact-Us page (‘http://localhost:3000/Contact-Us’)



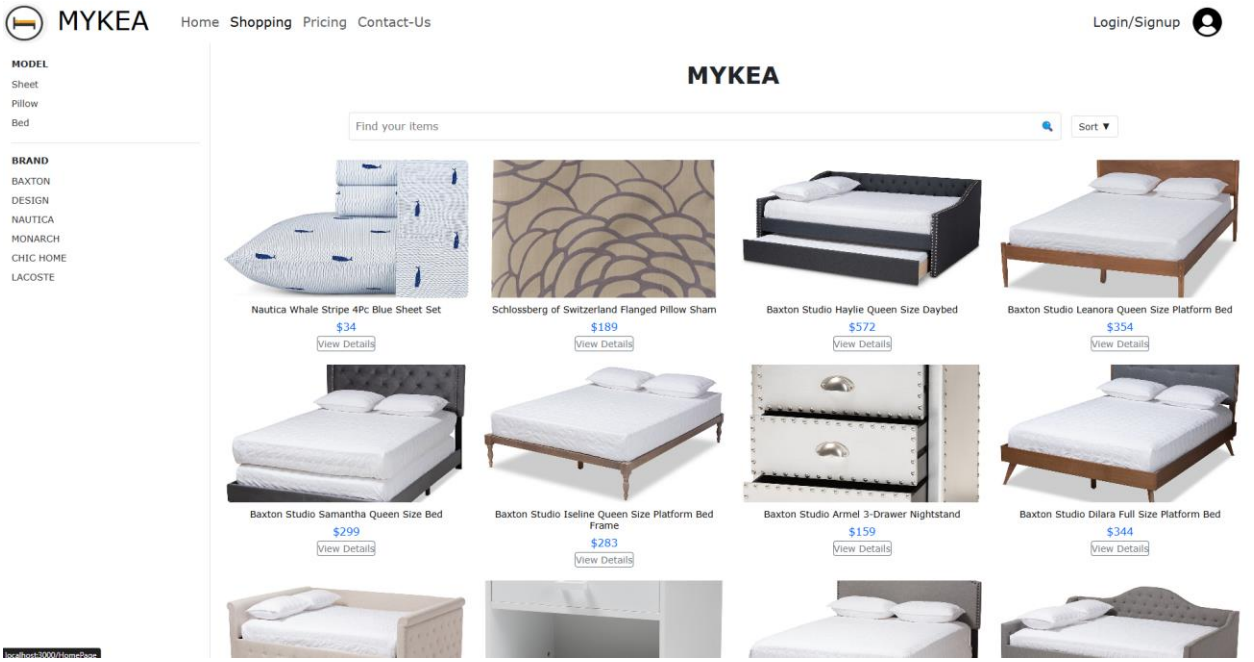
- ProductDetail page (‘http://localhost:3000/ProductDetail’)



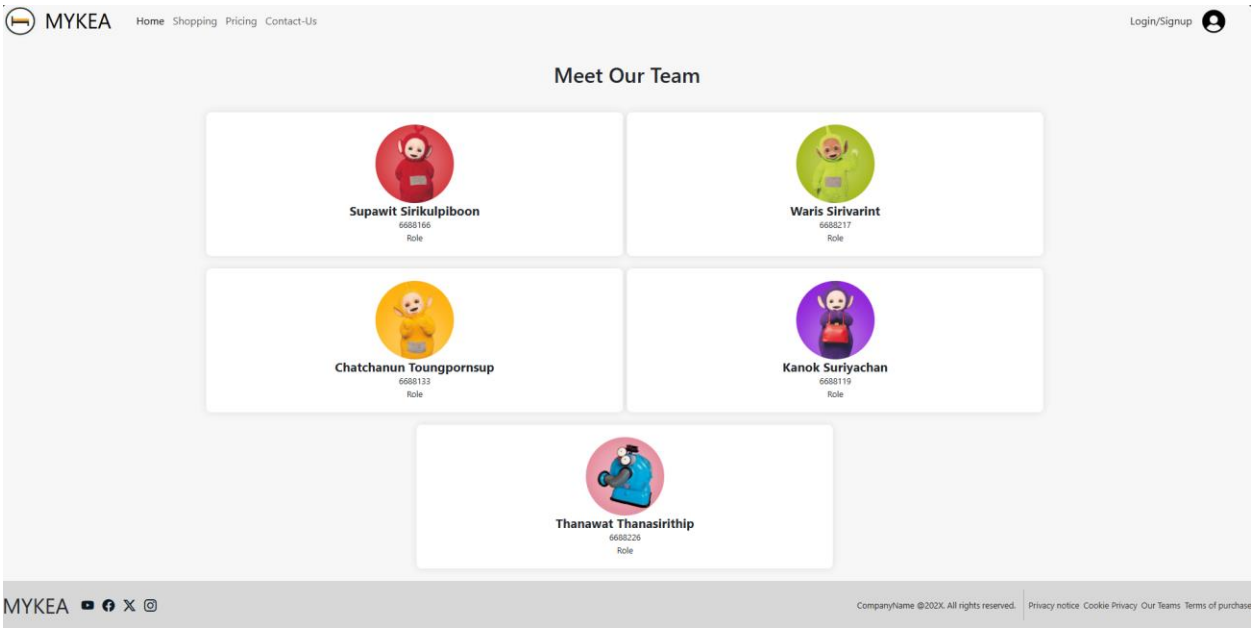
- ProductManagement page (‘http://localhost:3000/ProductManagement’)



- Search page (‘http://localhost:3000/SearchPage’)



- Team page (‘http://localhost:3000/OurTeam’)



References

https://ellebeddingth.com/th/

https://www.potterybarn.com/shop/bedding/

https://www.marksandspencer.com/l/home-and-furniture/bedding